



UNIVERSIDAD DE MÁLAGA



Máster en Ingeniería del Software e Inteligencia Artificial

Automatización de revisiones en integración continua Automation of reviews in continuous integration

Realizado por
Zoilo Hernández Duarte

Tutorizado por
Gabriel Jesús Luque Polo

Departamento
Lenguajes y Ciencias de la Computación
UNIVERSIDAD DE MÁLAGA

MÁLAGA, septiembre de 2025

ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA

MÁSTER UNIVERSITARIO EN INGENIERÍA DEL SOFTWARE E
INTELIGENCIA ARTIFICIAL

Automatización de revisiones en integración continua

Automation of reviews in continuous integration

Realizado por

Zoilo Hernández Duarte

Tutorizado por

Gabriel Jesús Luque Polo

Departamento

Lenguajes y Ciencias de la Computación

UNIVERSIDAD DE MÁLAGA

MÁLAGA, septiembre de 2025

Automatización de revisiones en integración continua

Zoilo Hernández Duarte

Resumen

El uso de la Integración Continua y el Despliegue Continuo (CI/CD) en la actualidad ha influenciado en gran medida los procesos software contemporáneos; promoviendo la agilidad de desarrollo y despliegue, así como la automatización de las tareas. Sin embargo, las revisiones de programas y código constantes sobre las distintas iteraciones del software son una de las partes que siguen planteando desafíos de optimización con respecto al tiempo invertido en ellas. Así, en este proyecto planteamos un sistema de optimización de archivos basado en la optimización multiobjetivo de los errores obtenidos en conjuntos de pruebas y cambios introducidos en un documento; utilizando para la generación del mismo las diferencias entre dos versiones del mismo código. El sistema resultante muestra resultados positivos en casos individuales a baja escala y presenta potencial para su expansión a casos con grandes cantidades de divergencias entre versiones.

Palabras clave: integración continua, reparación automática, optimización multiobjetivo, reparación de programas, algoritmos genéticos

Abstract

The use of Continuous Integration and Continuous Deployment (CI/CD) nowadays has greatly influenced contemporary software processes, promoting development and deployment agility as well as task automation. However, constant program and code reviews across different software iterations remain one of the areas that continue to pose optimization challenges regarding the time invested in them. Thus, in this project we propose a file optimization system based on multi-objective optimization of the errors obtained in test suites and the changes introduced in a document, using as input the differences between two versions of the same code. The resulting system shows positive results in small-scale individual cases and demonstrates potential for expansion to scenarios with large amounts of divergences between versions.

Keywords: continuous integration, automatic repair, multi-objective optimization, program repair, genetic algorithms

Índice

1. Introducción	5
1.1. Objetivos	6
1.2. Organización del documento	7
2. Revisión de la Literatura	7
2.1. CI/CD y revisiones de código	8
2.2. Reparación de programas automatizada	8
2.3. Reparación de programas automatizada basada en pruebas y programación genética	8
2.4. Herramientas para el uso de metaheurísticas y algoritmos	9

3. Descripción del acercamiento	10
3.1. Representación de una solución	11
3.2. Evaluación de una solución candidata	14
3.3. Algoritmos de optimización multiobjetivo	15
3.3.1. NSGA-II	16
3.3.2. SPEA2	17
3.3.3. Random Search (Búsqueda Aleatoria)	18
3.4. Operadores utilizados	18
3.5. Implementación	19
4. Diseño Experimental	20
4.1. Instancias de prueba	20
4.1.1. Calculator	21
4.1.2. Chart 22	21
4.1.3. Math 64	22
4.1.4. Time 3	22
4.1.5. Math 92	22
4.1.6. Lang 46	22
4.1.7. Math 21	23
4.1.8. Archivo combinado	23
4.2. Parámetros de los algoritmos	23
4.2.1. Tamaño de la Población, Iteraciones y Evaluaciones	24
4.2.2. Probabilidades de los operadores	25
4.2.3. Ejecuciones independientes	25
4.3. Métricas utilizadas	26
4.3.1. Hipervolumen (Hypervolume, HV)	26
4.3.2. Spread	26
4.3.3. Tiempo de ejecución	28
5. Análisis de los Resultados	28
5.1. Experimento 1: Ficheros individuales	28
5.1.1. Análisis por bloques	28
5.1.2. Análisis por líneas	30
5.1.3. Comparación	31
5.2. Experimento 2: Fichero combinado	32
5.2.1. Análisis por bloques	32
5.2.2. Análisis por líneas	33
5.2.3. Comparación	34
6. Conclusiones y trabajos futuros	34
6.1. Resumen del trabajo realizado	34
6.2. Líneas futuras	35

1. Introducción

En el campo de la ingeniería del software, el uso de la Integración Continua y el Despliegue Continuo (CI/CD por sus siglas en inglés) son dos prácticas de gran importancia en el contexto de la entrega rápida y fiable de productos software, acelerando el ciclo de vida del software y asegurando una mayor confiabilidad en la entrega [12, 13, 7], especialmente en tiempos más recientes. El uso de estos procesos lleva a la automatización de actividades clave como la construcción, las pruebas y el despliegue del software, todo ello con el objetivo de integrar cambios de manera eficiente y de detectar posibles errores lo más temprano posible [8]; aunque ello no conlleva la ausencia de desafíos como la dificultad en la visibilidad de la cadena de CI o defectos en la fiabilidad o seguridad en este proceso [23].

Sin embargo, a pesar de todos los aspectos positivos y mejoras que se intentan realizar con esta cadena de procesos; uno de los aspectos que todavía necesita de un considerable esfuerzo humano es la revisión y validación de modificaciones sobre el código, una tarea demandante en términos de tiempo y esfuerzo cognitivo [1]. La automatización de las pruebas puede ayudar a detectar regresiones que hayan ocurrido en el software, pero esta capacidad no implica la garantía de que se elija siempre la versión más adecuada de un código cuando existen múltiples alternativas para representar la versión final, cada una con características distintas y diferentes posibles fallos. Esto puede conllevar retrasos en el tiempo y reduce la eficiencia del entorno de desarrollo continuo.

Por otro lado, la investigación realizada en el campo de la reparación automática de software ha demostrado el potencial de usar enfoques basados en la búsqueda y el aprendizaje automático con el objetivo de generar y validar las posibles modificaciones de un programa [20] mediante distintas técnicas. Podemos encontrar así herramientas como GenProg [16, 24], ARJA [27] (creada con una fuerte inspiración en GenProg); y otros frameworks similares, que hacen uso de algoritmos evolutivos genéticos y evolutivos o alternativas como Nopol [25] con la resolución de restricciones; en estos se intenta aplicar sus técnicas a la creación de parches aplicados a un código original, el objetivo de los cuales sería hacer que el software supere un conjunto de pruebas, que anteriormente es posible que fallara. Así, la existencia y aplicación de estos enfoques sugiere un potencial considerable de los mismos para reducir el esfuerzo manual en la revisión de código y habilitar nuevos caminos a la automatización inteligente en los procesos de CI/CD. No obstante, este tipo de enfoque puede encontrarse a retos a la hora de intentar aplicar una búsqueda eficiente, como el sobreajuste de los posibles parches a las pruebas existentes [26].

Habiendo explorado estos trabajos y procesos, la relativa originalidad que se intenta integrar en este proyecto consiste en intentar tratar el problema desde un punto de vista distinto; no se busca generar un parche para un archivo desde cero; como podría ser contando solo con una instancia del mismo o con dos alternativas claramente diferenciadas como **correcta** e **incorrecta** necesariamente; la reparación se plantea como un problema en el que se deben explorar las posibles combinaciones de archivos alternativos. Este planteamiento, resulta cercano así a las prácticas en un flujo de trabajo de CI/CD, donde suelen crearse distintas variantes de un mismo código en paralelo, las cuales deben ser posteriormente integradas.

Por lo tanto, la idea principal consiste en lidiar con la cuestión de la revisión de código en unos términos iguales a los de un problema de búsqueda, donde las diferencias en fragmentos (o líneas) de código entre las alternativas de los archivos pueden elegirse o

combinarse de distintas formas, para dar lugar a una evaluación posterior del documento resultante ante una batería de pruebas implementadas en JUnit.

Para impulsar este planteamiento, el presente proyecto integra como recursos esenciales el uso de metaheurísticas (como NSGA-II [4] o SPEA2 [28]) para problemas de optimización multiobjetivo, estas técnicas nos asistirían con la selección automática de la mejor versión de un archivo al contar con dos alternativas de un mismo código con posibles defectos y errores en cada uno de ellos. Gracias a las características de este tipo de técnicas, se podría no solo minimizar los fallos de un programa, sino también optimizar otros elementos relevantes si fuera necesario, como el número de cambios aplicados. Para la implementación de estas metodologías en un código Java, se utiliza la biblioteca *jMetal* [6], un marco orientado a la resolución de este tipo de problemas.

En definitiva, la aproximación que deseamos realizar, intenta no solo explorar el potencial de los algoritmos evolutivos en la generación de soluciones válidas en un contexto de reparación de archivos, sino que también intenta contribuir a expandir el conocimiento sobre la integración de técnicas de optimización basadas en búsqueda en el contexto de procesos CI/CD. Se busca por ello, intentar medir la viabilidad y posibles ventajas de la utilización de estos métodos en la automatización de revisiones, con el objetivo de intentar simplificar y mejorar el proceso de cambio constante que implica la práctica de los procesos de CI/CD.

1.1. Objetivos

De esta forma, la idea principal del proyecto es intentar tratar la selección y combinación de distintas partes del código como si fuera un problema de búsqueda. Inicialmente se cuenta con dos versiones de un archivo, las cuales son comparadas: las diferencias entre ambos se organizan y se dividen en fragmentos discretos los cuales son puestos en un proceso de selección y recombinación para crear las versiones candidatas a ser finales. Como se ha mencionado, estos candidatos se ejecutan en una batería de pruebas escritas en el marco de JUnit, pudiendo utilizar los resultados de este proceso en la ejecución de un algoritmo genético implementado con *jMetal*, el cual realiza un proceso iterativo de evolución para identificar la combinación de fragmentos que genere como resultado el mejor archivo posible en el contexto de su proyecto con respecto a su grado de corrección, comportamiento y la cobertura de sus pruebas.

En conjunto, se crearía así un sistema que, con dos iteraciones de un mismo documento, podría encontrar una versión que refleje el contenido y comportamiento idóneo que se espera de él de forma automática. Si queremos reflejar las metas que perseguimos de forma explícita, podemos estructurarlos según el objetivo general principal, el cual se precisa del cumplimiento de tres objetivos parciales:

- **Objetivo general:** Diseñar, implementar y evaluar un prototipo que utilice algoritmos evolutivos para automatizar la generación de una versión óptima de un archivo fuente a partir de dos iteraciones previas, contribuyendo a reducir el esfuerzo manual en las revisiones de código. Para ello se necesita cumplir con otras metas parciales:
 - **Diseñar e implementar un prototipo** de un sistema que sea capaz de generar y evaluar un archivo resultado de la combinación de las diferencias entre dos versiones de un documento (seleccionando secciones de uno o de otro en cada punto de divergencia)

- **Aplicar algoritmos evolutivos** con la asistencia de *jMetal* aplicando los principios de un problema de búsqueda y optimización para encontrar un archivo óptimo con respecto a las pruebas definidas.
- **Comparar la eficiencia y efectividad de distintas estrategias y algoritmos** sobre distintos tipos de problemas y archivos, mediante el análisis de los resultados obtenidos gracias a métricas relevantes obtenidas durante el proceso.

Con este trabajo, se espera poder contribuir a la investigación que se sigue haciendo en la actualidad sobre la automatización en la ingeniería de software, concretamente, sobre el potencial de la implementación de métodos de optimización basados en búsqueda en los procesos CI/CD con los que podría reducirse el esfuerzo manual necesario para las revisiones de código así como incrementar la fiabilidad de las decisiones automatizadas en estos contextos.

1.2. Organización del documento

En este documento, recogemos así la información relevante y organizada del proyecto y la estructuramos en los siguientes capítulos:

1. **Capítulo 1:** un capítulo de introducción estableciendo el contexto del trabajo así como las tareas a nivel general que se deben realizar y los objetivos cumplidos con ellas.
2. **Capítulo 2:** segmento de Revisión de la Literatura en el que se exploran los trabajos previos realizados en los campos relacionados con el proyecto a implementar.
3. **Capítulo 3:** donde se describe el planteamiento del problema, así como las técnicas utilizadas en este contexto para solucionarlo.
4. **Capítulo 4:** una descripción del experimento establecido con el sistema implementado, incluyendo las instancias, parámetros y métricas que se utilizarán.
5. **Capítulo 5:** capítulo donde se reflejan los resultados obtenidos y se analizan con respecto a las métricas obtenidas.
6. **Capítulo 6:** discusión del proyecto y las posibles líneas de trabajo basándonos en el resultado de este.

2. Revisión de la Literatura

En esta sección se realiza una revisión de los trabajos de investigación y tecnologías relacionadas con los campos relevantes a nuestro proyecto; como la automatización de la reparación de programas y el uso de técnicas de búsqueda en este ámbito, con el objetivo de intentar poder asistir en el contexto de procesos CI/CD. Concretamente, queremos explorar brevemente los aspectos más relacionados con el proyecto y alguno de los trabajos que los tratan, mayormente, aquellos cuya previa examinación asista en la orientación y creación del proyecto.

2.1. CI/CD y revisiones de código

Las prácticas de CI/CD han alterado los métodos de desarrollo y despliegue de software, fomentando ciclos de integración rápidos y reduciendo riesgos en la entrega de nuevas versiones [12, 13, 7]. Al mismo tiempo, también surgen desafíos relevantes: como las faltas de visibilidad, fiabilidad o seguridad en las cadenas de procesos de este tipo [23, 8].

Incluso en este contexto, las revisiones de código pueden ser costosas en cuestión de tiempo y esfuerzo dependiendo de la doctrina seguida [1]; entre otras cosas, es probable que se de la necesidad de obtener una comprensión del programa amplia para revisar el código de forma efectiva que previamente no se tenía, e incluso puede ser necesario establecer comunicación con el autor del mismo (al mismo tiempo también se reconocen ciertos beneficios de la actividad, como el encontrar soluciones alternativas o mejorar el estilo del código). Por ello, la automatización de estas actividades podría intentar reducir el tiempo y esfuerzo dedicado a ellas en entornos CI/CD, al mismo tiempo que se intentan mantener sus posibles beneficios.

2.2. Reparación de programas automatizada

La reparación de programas automatizada es un campo en el que, como implica su nombre, se intenta generar parches y encontrar soluciones para errores en programas software de forma automática. En este sentido, podemos referirnos a trabajos como [20] que reflejan una perspectiva general sobre el campo; describiéndolo como un ámbito con orígenes en la investigación de los años 70 y 80, pero cobrando también importancia en décadas más recientes, empezando a explorar ideas muy importantes como el cambio automático de programas (como *behavioral repair*) en tiempos más recientes. Señala, adicionalmente, la necesidad de actualizar las técnicas existentes al tamaño y escala del software actual y, por tanto, es un campo que en muchos aspectos se beneficiará de una mayor atención y profundidad en su investigación.

Dentro de este contexto, es importante señalar concretamente la Reparación de Programas Automatizada basada en pruebas, que cuenta con distintas técnicas y herramientas, como podemos ver en [19]; en estas, la validez de los programas software y sus parches se corrobora con un conjunto de pruebas. Con ello, podemos utilizar tests para medir la validez de nuestros programas y medir la competencia de los parches generados automáticamente.

2.3. Reparación de programas automatizada basada en pruebas y programación genética

Concretamente, algunos de los sistemas más importantes y representativos de este tipo de reparaciones son las herramientas como GenProg [24, 16], RSRepair [22] o Arja [27] y métodos como Nopol [25] que demuestran algunos de los beneficios de tomar este camino en la automatización de las reparaciones, aunque por supuesto no sin inconvenientes u obstáculos en sus procesos, como la eficiencia o el sobreajuste [26]. Adicionalmente, en estudios y trabajos como [18] y [5], se refleja que la eficacia de estas herramientas puede ser inconsistente y variar de forma considerable debido a razones de naturaleza distinta como puede ser simplemente la falta de tiempo o una configuración errónea.

Un aspecto importante para muchas de estas herramientas es el uso de técnicas de programación genética y algoritmos genéticos; el uso de estas técnicas nos otorga distintas

posibilidades, una de las cuales sería la utilización de una optimización multiobjetivo. Así, el uso de algoritmos como NSGA-II [4] nos permitiría obtener un conjunto de soluciones que reflejara un compromiso entre distintas metas que se podrían querer obtener; en nuestro caso este principio podría aplicarse potencialmente para no solo intentar superar las pruebas preparadas; sino intentar optimizar otros aspectos del programa como la simplicidad; algo que beneficiaría a los procesos de CI/CD, ya que la mantenibilidad del código es de alta importancia en ellos.

2.4. Herramientas para el uso de metaheurísticas y algoritmos

La implementación de algoritmos de búsqueda en entornos prácticos requiere marcos que soporten personalización y análisis. Entre ellos destaca *jMetal* [6], un framework para Java diseñado para la resolución de problemas de optimización multiobjetivo (principalmente, aunque incluye otras opciones); *jMetal* proporciona implementaciones de metaheurísticas ampliamente utilizadas y facilita la recopilación de métricas relevantes para el análisis de resultados, lo que lo convierte en un recurso idóneo para proyectos experimentales como el presente. Además, trabajos recientes como [21] muestran cómo es posible utilizar *jMetal* para adaptar las configuraciones de un algoritmo a un problema para aumentar su eficiencia (utilizando en este caso una herramienta de configuración automática como *irace*), mostrando así su versatilidad.

Adicionalmente, *jMetal* no es la única herramienta de este tipo, existen una variedad de artefactos similares que varían según el lenguaje al que se aplican y los problemas que intentan solucionar. Primeramente contamos con una iteración de esta herramienta: *jMetalPy* es una versión de *jMetal* aplicada al lenguaje de programación Python, diseñada al igual que su antecesor, para lidiar con problemas de optimización multiobjetivo y con características como paralelismo o generación de métricas.

Después, podemos señalar a *ParadisEO*, un framework en C++ orientado a la creación modular y eficiente de metaheurísticas. Su módulo *ParadisEO-MOEO* proporciona soporte completo para optimización multiobjetivo mediante algoritmos como NSGA-II, SPEA2 o MOGA, junto con indicadores de calidad (hipervolumen, etc.) y arquitecturas paralelas y distribuidas [3, 17]. Por otra parte, *DEAP* es una herramienta en Python que ofrece la posibilidad de realizar actividades de prototipado rápido para algoritmos evolutivos, con estructuras de datos transparentes y herramientas para paralelización automática. De esta forma, resulta útil para el desarrollo de algoritmos evolutivos personalizados para las tareas que sean necesarios [10]. También contamos con con librerías como *pymoo*, más especializada en optimización multiobjetivo en Python que *DEAP* y orientada más al uso directo de los algoritmos y herramientas que ya posee, pero sin descuidar al mismo tiempo la personalización de algoritmos en caso de necesitarlo [2].

De esta forma, contamos con un amplio conjunto de herramientas que facilitan el uso de metaheurísticas y algoritmos multiobjetivo (y de otros tipos) que permiten implementar la teoría tras su funcionamiento, en los contextos de las grandes cantidades existentes de lenguajes software en la actualidad y todas las posibilidades que se pueden dar a la hora de resolver estos tipos de problemas.

En definitiva, mediante una revisión del trabajo realizado en estos campos hasta el momento podemos establecer varios puntos:

- La reparación de programas automatizada es un campo de interés y amplio, siendo la reparación basada en pruebas una variación de este principio con solidez pero problemas de escalabilidad, sobreajuste y una dependencia en la calidad de los tests.

- Es de interés intentar aplicar reparaciones automáticas en entornos CI/CD para aumentar la eficiencia en las revisiones de los programas y aumentar la fiabilidad de los procesos.
- La aplicación de algoritmos evolutivos y metaheurísticas multiobjetivo como NSGA-II en las reparaciones automáticas otorga flexibilidad a la hora de explorar múltiples soluciones y la posibilidad de optimizar más de un objetivo en ocasiones.
- Existen frameworks y herramientas como *jMetal* que permiten la implementación de este tipo de estrategias en programas software y la configuración optimizada de estos.

Con estos conceptos podemos así estructurar y orientar la forma de realizar el trabajo para conseguir las metas que queremos conseguir con el proyecto: la creación de un sistema que combine estos elementos para obtener la mejor versión de un programa o archivo software.

3. Descripción del acercamiento

De esta forma, en este proyecto, deberemos encontrar una solución a la cuestión de la automatización en la creación de una versión óptima de un programa o archivo software en el lenguaje de programación Java, basándonos en la existencia, propiedades y diferencias entre otras versiones previas desarrolladas durante un trabajo continuo y sus resultados en un conjunto de pruebas. Para ello, intentaremos plantear este tema como un problema de **optimización multiobjetivo basado en búsqueda**, en otras palabras, un problema de búsqueda en el que utilizaremos técnicas de optimización multiobjetivo que nos permitan perseguir de la manera más óptima una serie de metas. Básicamente, estos tipos de algoritmos se utilizan en problemas de optimización en los que se intenta maximizar o minimizar un conjunto de funciones objetivo que se aplican sobre las posibles soluciones del mismo, en ocasiones estando sujetos a una serie de restricciones.

De forma previa, en una instancia de este tipo de problemas contaremos con al menos dos factores importantes:

- Un par de archivos que representan dos alternativas de un mismo documento.
- Un conjunto de tests que realiza una serie de pruebas sobre este documento y sus funciones.

Señalar también, que dependiendo del caso, es posible que sean necesarios, por supuesto, una serie de archivos adicionales que sean necesarios para el funcionamiento correcto del par de archivos que se inspeccionan, pero que a priori no influyen demasiado en el proceso de optimización que nos concierne.

Por lo tanto, primeramente, debemos establecer las metas que queremos llegar a conseguir a la hora de construir un documento final. El primero de ellos resulta relativamente sencillo de establecer teniendo en cuenta las condiciones y factores establecidos: queremos que el documento creado como resultado de nuestro proceso sea lo más correcto posible, una medida que obtendríamos a partir de su ejecución ante una batería de pruebas preestablecida. Por ello, uno de los objetivos que estableceremos será **minimizar el número de tests que se fallan** durante la ejecución de las pruebas, lo cual indica que el funcionamiento del programa es el debido.

Aunque podemos considerar este objetivo como nuestra principal meta, es probable que en muchos casos existan distintas opciones a la hora de minimizar el número de errores; el mero hecho de intentar eliminar directamente las partes que introducen errores sin intentar modificarlas podría hacer superar estas pruebas, pero este curso de acciones no es tan conveniente y podría ser simplemente una acción que busque el pasar las pruebas pero no mejorar o arreglar el programa software. Esto nos lleva a plantear un segundo objetivo a perseguir en el proceso.

Establecer un segundo objetivo a seguir es algo más complicado, puesto que existen distintas posibilidades para ello. Por ejemplo, podríamos establecer que el código resultante fuera lo más corto posible, para mantener la funcionalidad deseada y minimizar el código innecesario, o también sería posible intentar incluir más o menos versiones de los cambios de una de las alternativas, es decir, elegir el mayor número de cambios de parte de una de las versiones y evitar la otra lo máximo posible.

Esta última perspectiva, puede darnos un factor interesante si le aplicamos un punto de vista muy típico en los entornos CI/CD como hemos establecido (y en general también aplicable a la mayoría de procesos software). En estos, los archivos son constantemente revisados y cambiados, comprobando su funcionamiento con pruebas como hemos establecido; sin embargo, en ocasiones el intentar arreglar los errores de un archivo puede traer consecuencias inadvertidas: muchas veces los errores pueden no ser resueltos correctamente o incluso el modificar los archivos lleva a la aparición de otros comportamientos no deseados y por tanto generar más errores, lo cual implica que el parche aplicado sobre el archivo original no era de alta calidad.

Así, podemos intentar perseguir obtener una medida de la calidad de un parche si tomamos el par de archivos como una iteración sobre el mismo documento: uno sería el intento de reparo del otro. Para ello, podemos establecer como segundo objetivo a minimizar el número de secciones o líneas del archivo que consideremos como "previo" que se incluirán en el resultado: de esta forma se persigue al mismo tiempo que se pasen correctamente el mayor número de pruebas posibles, manteniendo la mayor parte de los nuevos cambios introducidos en la modificación del código. Si durante la búsqueda se obtiene una solución con ambos valores cercanos al mínimo posible (en muchos casos 0) se establecerá que el parche ha sido correcto, pero si no es posible encontrar una solución óptima para ambos objetivos, es posible que el parche sea no del todo correcto o incluso haya perjudicado al funcionamiento del programa con fallos y errores nuevos.

Una vez establecidas las metas, debemos establecer los siguientes pasos del funcionamiento teórico de nuestro proyecto: primero deberemos encontrar las formas de codificar y representar los documentos resultado de forma que puedan ser identificados, procesados, diferenciados y evaluados por el algoritmo que se utilice; posteriormente, debemos especificar el algoritmo principal que se usarán durante el proceso así como posibles alternativas para una posible comparación y por último elegir también los operadores que se usarán en conjunción con el algoritmo. En definitiva, debemos establecer las especificaciones del proceso de búsqueda que guiarán el resultado a obtener.

3.1. Representación de una solución

Quizás el elemento más importante a seleccionar es la elección de la metodología a la hora de representar las soluciones candidatas (los archivos finales); esta elección determina en gran medida el funcionamiento del proceso de búsqueda y condiciona alguno de los otros factores que debemos determinar posteriores a ellos.

Así, como hemos establecido, en un caso básico de este problema contaremos con dos versiones de un documento, y un conjunto de pruebas sobre ambas. Estos archivos coincidirán parcialmente en su contenido, pero tendrán ciertas secciones en la que diferirán parcial o completamente. La presencia de estos cambios, sus posibles combinaciones y sus consecuencias, son los principales factores de interés en nuestro proyecto, y son los que influyen los objetivos que perseguimos; por lo tanto, a la hora de estructurar y codificar las representaciones de las soluciones candidatas durante la búsqueda, debemos encontrar una forma que traduzca de una forma correcta todas las versiones del archivo que se pueden generar con los distintos cambios.

Teniendo en cuenta algunas de las técnicas exploradas en el Capítulo 2, y observando el hecho de que lidiamos con dos versiones de un archivo donde debemos seleccionar los segmentos de código modificado del fichero original o del modificado, podríamos intentar aplicar una codificación binaria sobre las soluciones del problema. En esta metodología, las soluciones se representan mediante un vector binario, es decir, una serie de dígitos binarios (solo toman valor 0 o 1); se utiliza en muchas ocasiones con Algoritmos Genéticos y en problemas en los que se deben tomar decisiones o realizar elecciones discretas. Al observar el funcionamiento de este tipo de representación, podemos ver como sus propiedades se pueden adaptar satisfactoriamente a las condiciones de nuestro problema y ello nos lleva a examinar cómo enlazarlos y utilizar esta representación.

Al fin y al cabo, en este problema la exploración entre distintas soluciones se realizará mediante la elección de incluir o no líneas o segmentos de diferencia de cada uno de los documentos, y la combinación de estas decisiones llevará a la generación de unos resultados u otros. Sin embargo, es necesario otro paso de estructuración de los cambios para poder reflejar los resultados en este tipo de estructura; para ello debemos mirar a los documentos en sí y como se tratan este tipo de diferencias entre ellos, algo que en los contextos software y de desarrollo CI/CD aparece con frecuencia mediante el uso de herramientas *Diff* [11, 15]; mediante las cuales se realiza un breve análisis de las divergencias entre dos documentos y se puede obtener un resumen rápido de los cambios realizados entre ellos.

Así, a la hora de comparar dos archivos, en términos prácticos, las mínimas unidades de diferencia que se suelen cuantificar al realizar una examinación entre dos programas son las líneas; las cuales pueden ser añadidas, eliminadas o modificadas entre alternativas. Por un lado, lidiar únicamente con líneas otorga un mayor grado de personalización y control sobre el resultado del archivo; sin embargo, a la hora de lidiar con documentos de mayor extensión puede resultar algo ineficiente, por ello existe otra medida muy común en este contexto que son los segmentos (o fragmentos) de diferencia, llamados comúnmente *chunks* los cuales, como su nombre indica, combinan varias líneas que son diferentes entre ambos archivos y forman un grupo representativo de divergencia.

Por ello, debemos establecer que tipo de organización de las diferencias seguiremos a la hora de relacionarlas con la estructura binaria. Inicialmente, se diseñó la aproximación basada en fragmentos, debido a una traducción más directa con el vector binario y la mejor eficiencia en archivos más grandes; una opción más sencilla y directa.

Para ello, construiremos un vector binario de una longitud que se corresponderá al número de deltas entre ambos documentos (siendo un delta una ocurrencia de diferencia en el texto entre ambas versiones). Cada bit de la cadena, representa la versión de ese delta que se reflejará en el resultado: si es un 0 se elegirá la versión del archivo “original”(o archivo 0 si se prefiere) y si es un 1 se elegirá la versión del archivo “nuevo”(o archivo 1 si se prefiere). Por lo tanto digamos que el bit de la n -ésima posición representa la elección

de la versión de inclusión del n-ésimo delta. Esta metodología se representa en la Figura 1. También en dicha figura podemos ver que esta representación funciona tanto en casos de cambios entre los fragmentos como eliminación/adición por completo.

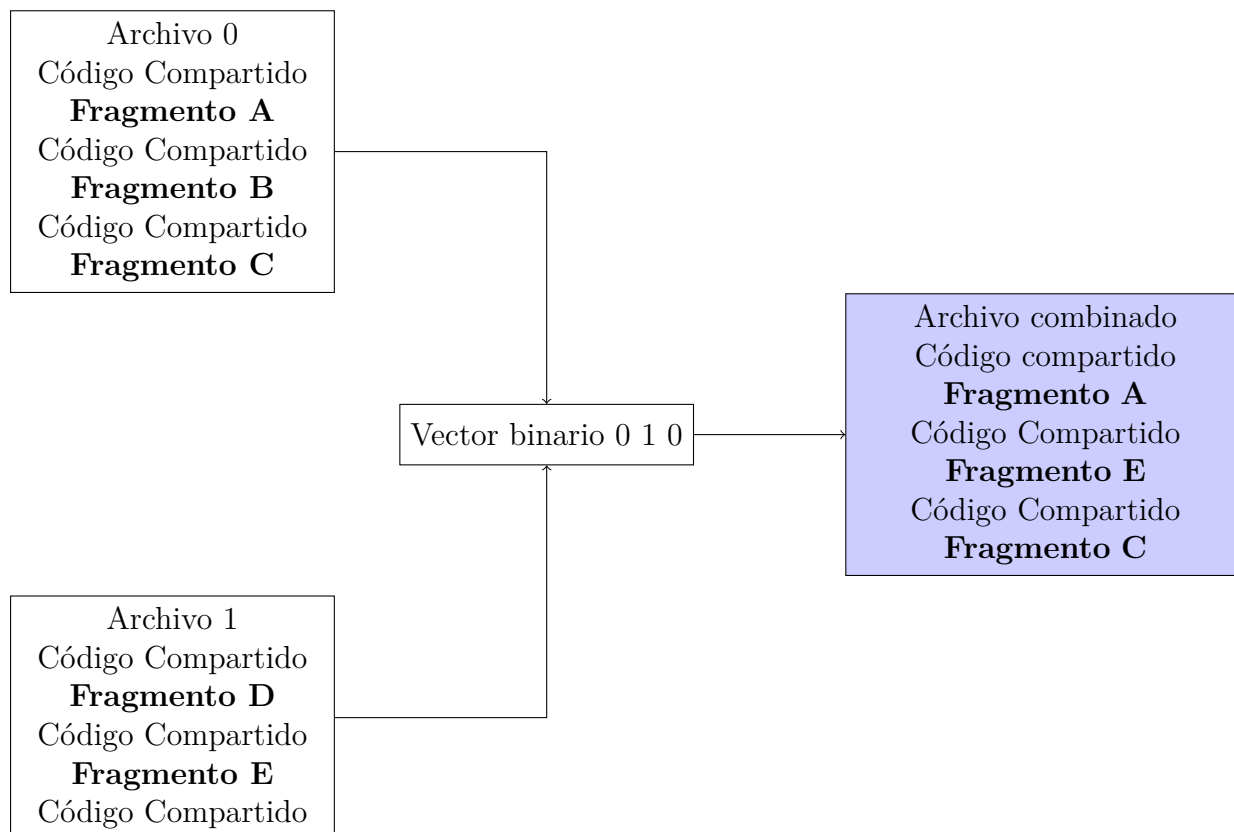


Figura 1: Representación de la selección de fragmentos a partir de un vector binario. Código compartido representa segmentos del documento que son iguales entre versiones.

Una vez que contamos con este acercamiento, implementamos una alternativa que lidiara con los documentos a nivel de líneas, el implementar el proceso de esta manera, se nos otorga una reparación mucho más específica y detallada pero se crea la necesidad de incrementar de forma muy significativa el espacio de búsqueda. Para usar esta teoría en conjunto con el vector binario, decidimos combinar estos dos factores con la existencia de los fragmentos de diferencia entre documentos; así, en este caso el vector contará con un número de dígitos igual al número de líneas combinadas entre todos los deltas de ambos archivos, en el orden de los deltas y dentro de cada delta primero incluyendo las líneas del archivo original y después las del archivo con “parche”. Por ejemplo, si el archivo 0 tiene dos fragmentos de diferencia con el archivo 1 con dos líneas en el primero y una línea en el segundo; mientras que el archivo 1 tiene también dos fragmentos pero con una línea cada uno, tendríamos como soluciones vectores binarios de cinco dígitos y sus valores representarían la no inclusión (un valor 0) o inclusión (un valor 1) de esa línea en el documento final. Este caso se puede ver en la Figura 2.

De esta forma, contamos con dos formas distintas de representar las combinaciones de código que se pueden dar entre dos archivos de forma discreta y controlada. El siguiente paso será establecer el proceso de evaluación de una solución que se utilizará durante la ejecución del algoritmo de optimización a utilizar.

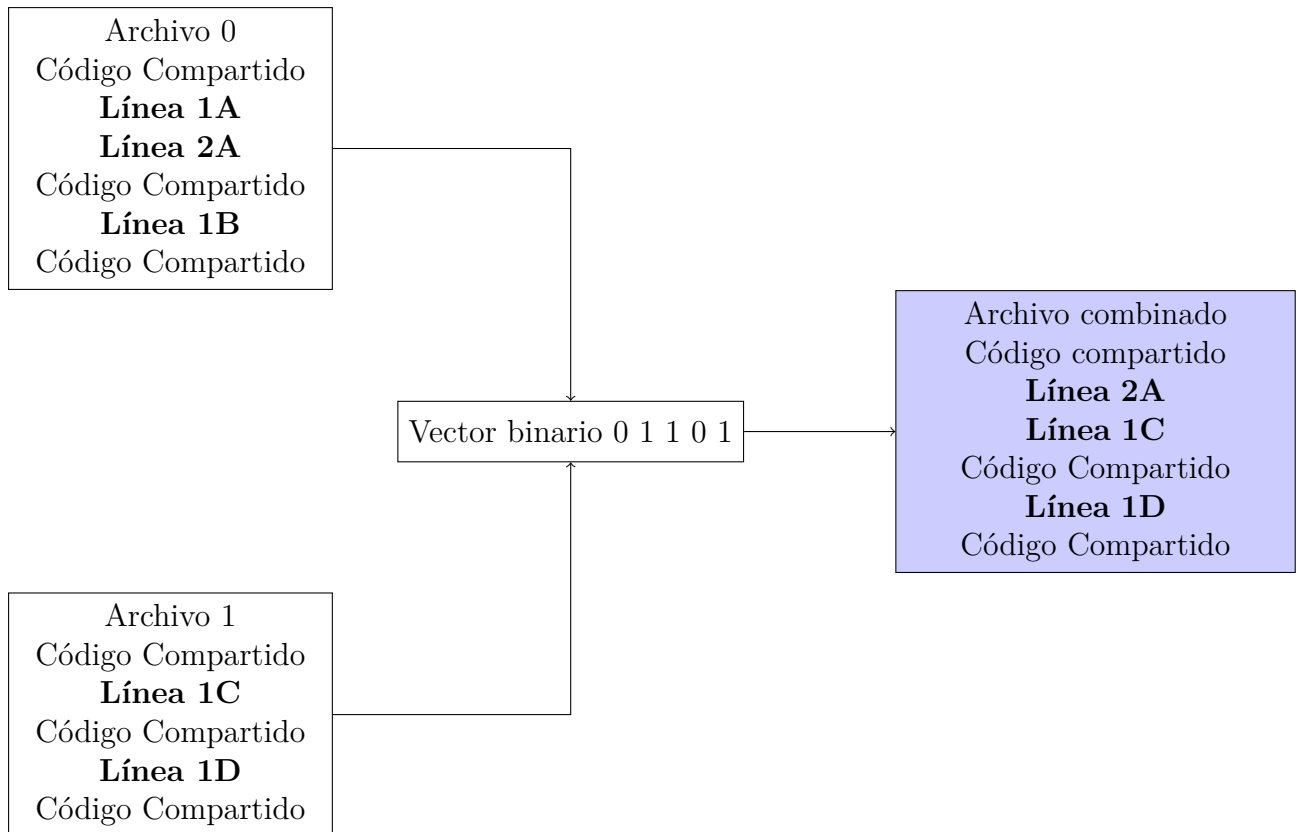


Figura 2: Representación de la selección de líneas a partir de un vector binario. Código compartido representa segmentos del documento que son iguales entre versiones. Línea 1A representa la primera línea del fragmento A

3.2. Evaluación de una solución candidata

Una vez se cuenta con una vector binario, será necesario realizar el proceso por el que se obtiene una evaluación del mismo. Para ello, tendremos que establecer los pasos a seguir, los cuales vienen altamente marcados por los objetivos que hemos establecido al principio del capítulo; en otras palabras, tendremos que obtener las medidas de los objetivos que queramos minimizar o maximizar para conocer el valor de la solución a partir de su vector binario. En este caso, los resultados que queremos obtener serán el número de pruebas superadas por la solución (con respecto a un conjunto concreto) y el número de secciones del archivo original.

Primeramente, dado el vector concreto, tendremos que construir el archivo resultado siguiendo una de las metodologías que hemos establecido en la sección anterior: por fragmentos o por líneas. Una vez construido, podemos obtener y guardar una de los objetivos que perseguimos: el número de segmentos de código que se eligen del archivo original y seguir con el proceso. Una vez hemos construido, es necesario compilar el archivo para su posterior inclusión y ejecución en el conjunto de pruebas de JUnit designado, donde obtendremos el otro objetivo a minimizar: el número de errores ocurridos durante la realización de dichas pruebas. Con este proceso obtenemos así la evaluación de una solución específica.

Sin embargo, existe una excepción concreta a la hora de obtener los objetivos de ciertas soluciones: se trata de aquellos casos en los que el documento final no compi-

le correctamente debido a errores de sintaxis o de otra naturaleza. Es natural, que al combinar fragmentos o líneas de dos versiones distintas de un archivo se produzcan en muchos casos contradicciones, omisiones o adiciones sin sentido entre los mismos; por ejemplo, puede ser común el no cerrar correctamente ciertas estructuras como bucles o condicionales o la utilización de variables que no han sido declaradas (como debe ser en Java). Esto además, es exacerbado al utilizar las líneas al construir los documentos, ya que contamos con mayor control de lo incluido en el mismo, pero al mismo tiempo una mayor posibilidad de error.

Por ello, debemos determinar qué hacer cuando se produce este caso; ya que la malformación del código u otros errores que lleven a este estado del documento no siempre producirá los mismos resultados en los objetivos, pero al mismo tiempo, es una situación que no se puede permitir: al empezar con archivos que como mínimo están bien formados y son ejecutables, es inadmisibles que una solución formada por el proceso de reparación, acabe con estos errores de compilación. Por lo tanto, para establecer estos casos como indeseables, los resultados que se le atribuirán a los objetivos de la evaluación serán los peores que se puedan dar en su contexto; en otras palabras, una vez que se ha comprobado que el archivo resultado tiene errores de compilación o similares, sus resultados son automáticamente establecidos como el mayor número de errores posibles en el conjunto de prueba y el mayor número de fragmentos o líneas del archivo original; esto hace que esas soluciones son siempre, al menos, igual de desfavorables que los peores casos construidos correctamente. En la Figura 3 podemos observar un diagrama de flujo que describe los pasos realizados brevemente.

Habiendo establecido la codificación y proceso de evaluación de las soluciones, tenemos proceso básico de representación y medición de los archivo resultado de las combinaciones de diferencias entre dos versiones anteriores. El siguiente paso, sería el establecimiento del algoritmo concreto que gestione y realice la búsqueda de las distintas soluciones, evaluándolas para encontrar aquella que se adapte mejor a nuestros objetivos.

3.3. Algoritmos de optimización multiobjetivo

Así, necesitamos determinar las técnicas y métodos que se utilizarán para explorar el espacio de soluciones que utilizarán las representaciones establecidas, evaluarlas individualmente y, eventualmente; encontrar la más óptima del espacio explorado. Como hemos podido ver durante la revisión de la literatura, la utilización de algoritmos evolutivos y genéticos es una práctica ya presente en el contexto de la reparación automática [27, 16] y ha contado con gran importancia en el mismo. Este hecho, combinado con la utilización de vectores binarios que se prestan fácilmente a este tipo de técnicas, nos lleva a perseguir la implementación de este tipo de algoritmos.

Más concretamente, haremos uso de dos de los algoritmos evolutivos de optimización multiobjetivo más utilizados: NSGA-II [4] y SPEA2 [28]. Haremos uso de ambos para poder realizar una comparativa sobre los resultados obtenidos por cada uno y la eficiencia al obtenerlos respectivamente; adicionalmente también haremos uso de la técnica de Búsqueda Aleatoria, también llamado Random Search (RS) para comprobaciones adicionales, ya que existen también herramientas como RSRepair [22] que realizan un uso satisfactorio de esta alternativa. Para ello podemos examinar brevemente el funcionamiento de las tres y diferenciarlas.

Desde un punto de vista inicial, los tres algoritmos lidiarán con una población de soluciones sobre la que iterarán para encontrar una solución que optimice los objetivos

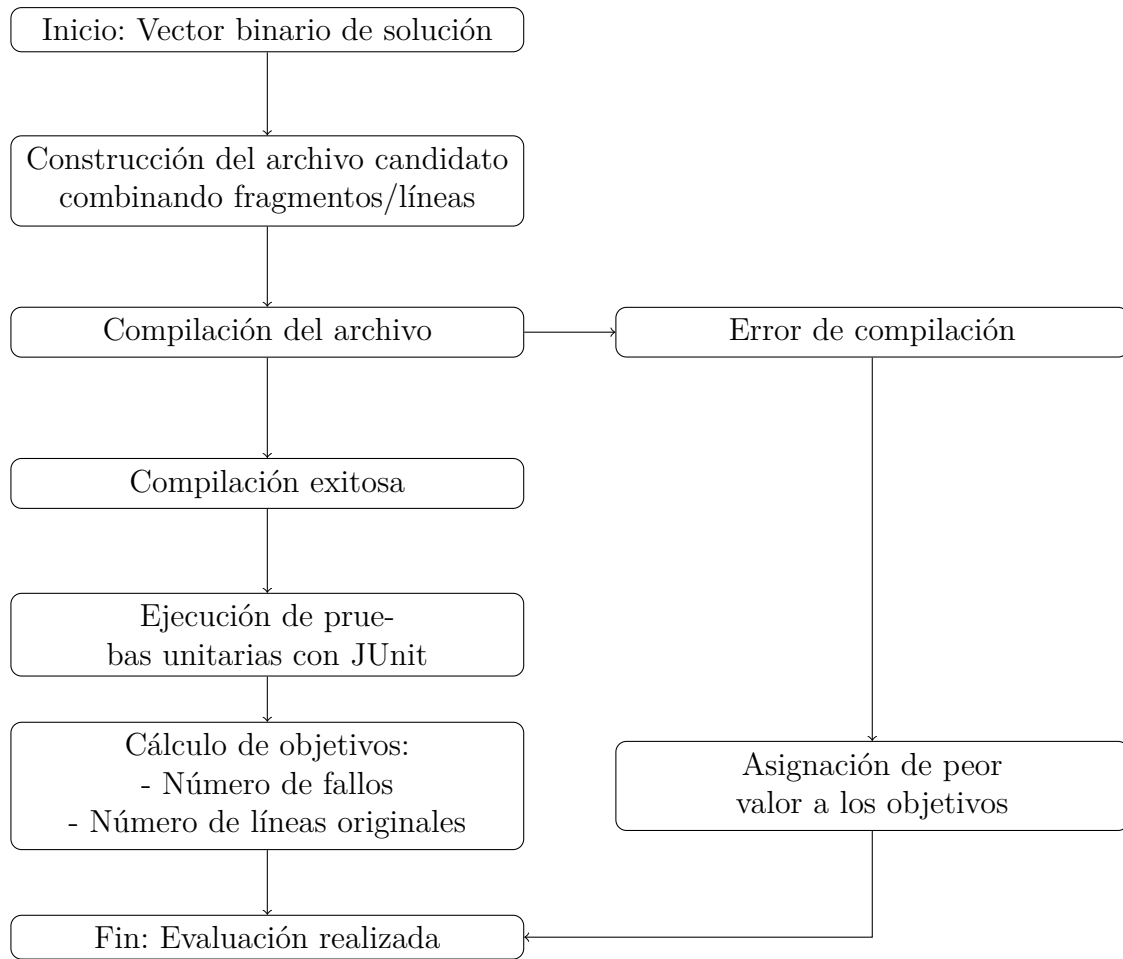


Figura 3: Diagrama de flujo de la evaluación de una solución candidata.

perseguidos; dentro de estos algoritmos multiobjetivo existe un concepto importante: la **dominancia de Pareto**. Este es un concepto que se aplica, en nuestro contexto, entre un par de soluciones e implica la comparación de ambas; específicamente, se dice que una solución domina a otra si en todos los objetivos es al menos igual de bueno y cuenta al menos con un objetivo en el que es estrictamente menor. Si hablamos de un problema de minimización y con dos objetivos como es nuestro caso, diremos que una solución **A** domina a otra **B** si los dos objetivos de A son menor o iguales a los de B y en al menos uno de los dos, es estrictamente menor, lo cual lleva a que en lo que corresponde a la evaluación de soluciones, A será considerada mejor que B.

Así, el objetivo de los algoritmos de optimización elegidos será encontrar el llamado **Frente de Pareto**, el cual es un conjunto de soluciones que no son dominadas por ninguna otra, es decir, no es posible mejorarlas con respecto a los objetivos que se persiguen y por lo tanto son el conjunto final que deseamos obtener para optimizar nuestras metas. Las metodologías utilizadas para llegar a este conjunto final de soluciones, serán los factores diferenciales entre los algoritmos utilizados.

3.3.1. NSGA-II

NSGA-II es el primero de los algoritmos escogidos, en gran parte por su amplio uso y popularidad. Su funcionamiento se basa en una población inicial P y las generaciones

que se producirán a partir de las mismas; se creará un P de un número concreto n y por cada miembro de P se creará una solución hijo a partir de la selección y recombinación de dos soluciones padre de P ; adicionalmente esta solución será alterada mediante un operador de mutación para mantener la diversidad genética y promover la exploración del espacio de búsqueda. Así, acabamos con dos conjuntos de soluciones: n padres y n hijos los cuales se unen en un solo grupo que debe ser procesado por los mecanismos de *ranking* y *crowding*, el primero genera una serie de frentes de Pareto ordenados de manera que el primero es un conjunto de soluciones no dominadas por ninguna otra, el segundo es dominado solo por las soluciones del primer frente y así sucesivamente; por otra parte, la distancia *crowding* nos da una medida de la diversidad que otorga una solución en su frente. Por lo tanto, la nueva población P será formada por las n soluciones obtenidas a partir de los frentes de Pareto ordenados, utilizando la distancia *crowding* para elegir dentro de un mismo frente en caso de que sea necesario; este proceso se repetirá a lo largo de un número deseado de generaciones. En el Algoritmo 1 podemos observar un segmento de pseudocódigo representativo.

Algorithm 1 NSGA-II

```

1: Inicializar población  $P_0$  con  $N$  individuos aleatorios
2: Evaluar miembros de  $P_0$ 
3: for generación  $t = 0$  a  $G - 1$  do
4:   Generar hijos  $Q_t$  con  $N$  individuos utilizando recombinación y mutación en  $P_t$ 
5:   Evaluar los miembros de  $Q_t$ 
6:    $R_t \leftarrow P_t \cup Q_t$ 
7:   Realizar ranking en  $R_t$  para obtener los frentes  $F_1, F_2, \dots$ 
8:    $P_{t+1} \leftarrow \emptyset$ 
9:   for cada frente  $F_i$  en orden do
10:    if  $|P_{t+1}| + |F_i| \leq N$  then
11:       $P_{t+1} \leftarrow P_{t+1} \cup F_i$ 
12:    else
13:      Calcular distancia crowding  $F_i$ 
14:      Seleccionar las  $(N - |P_{t+1}|)$  soluciones de  $F_i$  por distancia crowding
15:      break
16:    end if
17:  end for
18: end for
19: return población final  $P_G$ 

```

3.3.2. SPEA2

Por otro lado, el algoritmo SPEA2 (una mejora del algoritmo SPEA) centrará su actividad en la combinación de las poblaciones de soluciones que serán generadas con un archivo o conjunto separado, en el que se guardarán las soluciones de mayor calidad. Así, se contará con una población inicial P de N individuos aleatorios y un archivo externo A (conjunto que alberga soluciones o individuos) vacío con una capacidad determinada; en cada iteración se empieza evaluando (en base a los objetivos) los miembros de la población y se unirán las soluciones de la población y el archivo actuales para contar con todas las posibles soluciones en ese momento; de este conjunto combinado se obtendrán todos los miembros no dominados y se creará una nueva versión de A con las mejores soluciones

del grupo. Sin embargo, es posible que en esta nueva versión se haya sobrepasado la capacidad, en cuyo caso, se aplicará un truncamiento mediante el uso de la mínima distancia, es decir, se eliminarán del archivo aquellos que más cerca estén de sus vecinos (en caso de empate se escogerán las segundas mínimas distancias y así hasta satisfacer las necesidades); por otra parte, si la capacidad del archivo no se ha completado se podrá rellenar con soluciones dominadas del conjunto combinado.

En este punto si se ha llegado al número de generaciones establecido o a una condición de parada se podrá devolver el archivo como el conjunto de soluciones final, pero en caso contrario se realizará una selección de los individuos de A y se aplicarán recombinaciones y mutaciones para obtener la siguiente población, volviendo a repetir el proceso. Básicamente, se irá iterando una población y guardando sus mejores resultados en un archivo final que se usará como resultado. Un pseudocódigo de este funcionamiento se puede ver en el Algoritmo 2.

Algorithm 2 SPEA2

```

1: Inicializar población  $P_0$  con  $N$  individuos aleatorios
2: Inicializar archivo externo vacío  $A_0$  de capacidad  $C$ 
3: for generación  $t = 0$  hasta  $G - 1$  do
4:   Evaluar valores de los objetivos para  $P_t$ 
5:    $S_t \leftarrow P_t \cup A_t$ 
6:    $A_{t+1} \leftarrow$  individuos no dominados de  $S_t$ 
7:   if  $|A_{t+1}| > C$  then
8:     Aplicar truncamiento para eliminar elementos de  $A_{t+1}$ 
9:   else if  $|A_{t+1}| < C$  then
10:    Completar con las siguientes mejores soluciones dominadas
11:   end if
12:   Seleccionar padres de  $A_{t+1}$  usando torneo binario
13:   Aplicar cruce y mutación para generar  $P_{t+1}$ 
14: end for
15: return archivo final  $A_G$ 

```

3.3.3. Random Search (Búsqueda Aleatoria)

El último algoritmo que se utilizará será la búsqueda aleatoria; este es claramente más simple que las otras dos alternativas utilizadas: en su proceso no existe un grupo de soluciones ni la necesidad de crear nuevas generaciones a partir de las mismas; únicamente, a lo largo de un número de iteraciones establecido se generarán soluciones de forma aleatoria, se evaluarán y se irá guardando la mejor que haya encontrado hasta el momento; adicionalmente en casos como el nuestro guardará una lista de todas las soluciones que haya encontrado y no se dominen entre sí. De esta forma podemos aplicar de forma directa y sencilla esta metodología a modo de comparación y eficiencia básica.

3.4. Operadores utilizados

En el proceso de búsqueda y optimización de las soluciones, es necesaria no solo la elección de algoritmos, si no que debemos también establecer los tipos de operaciones que utilizarán. Estos elementos son típicamente los tipos de procesos que se realizan a la hora de guiar la creación de las nuevas poblaciones en los algoritmos evolutivos (y en

ocasiones de otros tipos de acciones). Con estos, siempre se intenta ajustar los procesos de búsqueda para que sean lo más eficientes posibles.

Los operadores más comunes y básicos en estos contextos son los de **selección**, **recombinación** y **mutación**; el primero decide qué miembros de la población pasarán a ser padres, la recombinación establece el modo en el que se combinará la información de las soluciones padres para crear a sus hijos y la mutación introduce pequeños cambios a la solución hija una vez que se ha creado. Estos tres, son así los que establecen las condiciones de la creación de las nuevas generaciones de soluciones y por tanto tienen gran influencia en el resultado final.

Así, en nuestro caso, estos tres operadores son los que se han establecido para intentar guiar correctamente los algoritmos de optimización:

- **Selección por torneo binario:** permite elegir soluciones candidatas para reproducción, seleccionando a dos individuos aleatorios de la población y escogiendo a la que mejor valor tenga para ser padre junto a otra (u otras); utilizando además la violación de restricciones como otro valor a la hora de la elección.
- **Cruce de un punto (Single-Point Crossover):** combina dos vectores binarios seleccionando un punto de corte y mezclando los genes de los padres a partir de dicho punto.
- **Mutación por bit flip (Bit-Flip Mutation):** invierte el valor de un bit con cierta probabilidad, explorando nuevas combinaciones de fragmentos.

3.5. Implementación

El prototipo del programa que se busca implementar se realizará en **Java** como se ha establecido y brevemente, hará uso de las siguientes herramientas para conseguir nuestros objetivos:

- El framework *jMetal* que nos permitirá definir el problema multiobjetivo, y utilizar sus definiciones y funcionalidades de los algoritmos pertinentes y sus operadores, además de obtener métricas de sus ejecuciones.
- La biblioteca **Java Diff Utils** para obtener las diferencias entre archivos y generar los parches de forma fácil
- **JUnit** para evaluar el grado de corrección de los archivos candidatos mediante pruebas unitarias.

Así, en el proyecto integraremos todos estos elementos para crear un sistema automatizado de selección y combinación de fragmentos y líneas de código, comprobado por un conjunto de pruebas y haciendo uso de distintos algoritmos para la optimización de los resultados. La combinación de estos procesos resulta en un grado de complicación e inversión temporal significativos, por lo que puede presentar obstáculos a la hora de establecer las condiciones y parámetros de los experimentos a realizar.

4. Diseño Experimental

Una vez hemos establecido el proceso por el que se guía la teoría del proyecto, debemos establecer las pautas a seguir para comprobar la efectividad, viabilidad y rendimiento del sistema construido a la hora de perseguir los objetivos establecidos. Para ello, debemos establecer una serie de experimentos que permitan analizar su comportamiento en distintos escenarios y contextos, de esta forma, podemos observar su comportamiento con distintos tipos de archivos y escenarios, una variedad que por supuesto es abundante en el campo del desarrollo software. Así tenemos que crear distintos casos en los que podamos crear distintas versiones de un documento combinatorio entre dos archivos alternativos del mismo y pasarlas por el mismo conjunto de pruebas para designar su grado de optimización en respecto a la minimización de fallos y cambios introducidos.

Para crear estos tipos de entornos de evaluación será necesario establecer tres elementos básicos para realizar las pruebas:

- **Instancias de prueba:** selección de archivos representativos, los cuales pueden ser resultado de desarrollo propio como de benchmarks o bancos de datos ampliamente utilizados en la literatura, que sirven como base para aplicar el enfoque en unos entornos realistas .
- **Parámetros de los algoritmos:** los algoritmos deben ser ajustados y configurados correctamente; elementos como el tamaño de la población del algoritmo, el número de evaluaciones totales, el número de iteraciones y las probabilidades de recombinación y/o mutación son factores de gran importancia y que pueden alterar el comportamiento del algoritmo de forma beneficiosa o perjudicial si no se realiza con cuidado.
- **Métricas de evaluación:** definición de los criterios utilizados para medir la calidad de las soluciones obtenidas, considerando tanto el cumplimiento de los tests como el número de fragmentos seleccionados y otras medidas relevantes.

Habiendo establecido estos factores, primeramente deberemos encontrar o elaborar los ejemplos con los que trataremos durante la experimentación; una vez escogidos podremos aplicar nuestros procesos y posteriormente podremos iterar sobre ellos así como modificar los parámetros para mejorar su eficiencia y obtener las métricas que deseemos

4.1. Instancias de prueba

Como hemos establecido, para intentar asegurar el funcionamiento correcto del sistema, debemos obtener los resultados de aplicarlos en documentos representativos de aquellos presentes en los procesos software a diario. Primeramente, se generó una instancia de prueba inicial llamada **Calculator** junto con su conjunto de prueba; las dos versiones de la calculadora han desarrolladas de manera propia y no han sido obtenidas de ninguna fuente externa; con el objetivo de disponer de una instancia controlada y relativamente simple como caso de estudio de partida.

Posteriormente, se hizo una selección de **seis archivos fuentes** que presentan distintas características en términos de tamaño, número de fragmentos divergentes y el conjunto de pruebas al que se someten. La mayoría de estos casos de prueba se han obtenido del conjunto de datos *Defects4J* [14], un *dataset* que recoge errores reales presentes

en versiones antiguas de software en Java los cuales ya han sido arreglados, por lo que se cuenta con los cambios que hacen que los programas software funcionen correcta e incorrectamente.

Por lo tanto, este es un conjunto de datos ampliamente usado en campos en los que se haga uso de **bugs** realistas, como es el pertinente en nuestro caso de reparación automática de programas y, además, cumple con nuestras condiciones de contar versiones alternativas (con y sin error), además de indicar los conjuntos de tests en los que se falla al añadir estos errores. Para hacer uso de estos archivos, crearemos versiones locales de los mismos y de sus tests, importando el comportamiento de sus bibliotecas que sea necesario para permitir su ejecución y sometimiento a pruebas; adicionalmente, aseguraremos que nuestras versiones se comporten igual a las de *Defects4J*, comprobando que los resultados de las versiones con errores y sin errores en el conjunto de pruebas sean idénticos a los del conjunto de datos original asegurando una duplicación de los comportamientos originales de los documentos.

De esta forma contaremos en total con siete casos de prueba:

- **Calculator** (Desarrollo Propio)
- **KeyedObjects2D** (*Chart 22*)
- **LevenbergMarquardtOptimizer** (*Math 64*)
- **MutableDateTime** (*Time 3*)
- **MathUtils** (*Math 92*)
- **StringEscapeUtils** (*Lang 46*)
- **RectangularCholeskyDecomposition** (*Math 21*)

Así, debemos explorar brevemente cada uno de estos con la información que necesitamos de ellos; extraída mediante las herramientas de implementación expuestas en el capítulo anterior:

4.1.1. Calculator

Calculator como su nombre indica, es un programa que recrea el comportamiento de una calculadora de forma muy básica mediante métodos como sumar, restar, multiplicar... Las versiones de los archivos varían ligeramente, pero tienen todos alrededor de 40 líneas de código físicas (LOC) y cuenta con 10 pruebas distintas en su conjunto de pruebas **CalculatorTest**. Adicionalmente, existen 9 fragmentos de código divergente entre ambas alternativas de la calculadora, que equivalen a 18 líneas en total si sumamos el número de líneas dentro de todos los fragmentos; concretamente, en este caso cada fragmento está formado por una única línea que suele ser la implementación de una simple operación. Esta calculadora, tiene un perfil algo distinto al de los siguientes casos, ya que la distribución de errores es algo más equilibrada entre versiones de archivo.

4.1.2. Chart 22

Chart 22 concierne a **KeyedObjects2D** una clase que, originalmente, forma parte de la librería **JFreeChart** de Java para la creación de gráficos en aplicaciones software.

Los documentos explorados del caso tienen una cuantía de LOC de alrededor de 430 líneas aproximadamente y el resultado es evaluado mediante 11 pruebas distintas en el documento `KeyedObjects2DTests`; entre ambas versiones del documento existen 5 fragmentos de diferencia con 40 líneas entre ellos. Este caso es el primero en el que los errores de compilación pueden ser comunes, ya que ciertas combinaciones pueden llevar a, por ejemplo, no cerrar correctamente estructuras que no existen en ambas alternativas, como condicionales o bucles.

4.1.3. Math 64

El siguiente *bug* en la lista sería *Math 64* el cual está presente originalmente en la clase `LevenbergMarquardtOptimizer`, perteneciente a la biblioteca *Apache Commons Math*, que asiste con componentes matemáticos o estadísticos en Java. Entre las versiones alternativas, se podría establecer una medida de LOC de entre 850 y 870 líneas, con un conjunto de pruebas en `MinpackTest` con 18 pruebas en total y concretamente, 2 de ellas fallando debido al error presente; por otra parte, según nuestras herramientas cuenta con 12 bloques de diferencia que incluyen en total 32 líneas. Cabe destacar un caso similar en *Defects4J: Math 68* que lidia con el mismo documento original y el mismo conjunto de pruebas, pero lo intenta arreglar de forma distinta.

4.1.4. Time 3

El caso *Time 3* está ligado a la clase `MutableDateTime` perteneciente a la biblioteca *Joda Time*, que permite asistir a la hora de trabajar y lidiar con fechas y tiempos de forma efectiva. Este archivo resulta algo más extenso al contar con unas 1470 líneas físicas de código (aunque cuenta con muchos comentarios) y el conjunto de tests que nos concierne se encuentra en `TestMutableDateTime_Adds`, generando este error fallos en 5 de las 40 pruebas totales en este banco, al mismo tiempo, contiene 10 bloques de diferencia entre las dos alternativas del documento, lo cual a su vez implica 40 líneas en total entre todos estas secciones.

4.1.5. Math 92

Math 92 es otro error sacado de la biblioteca *Apache Commons Math*, esta vez involucrando al archivo `MathUtils`, que posee unas 1100 líneas de código físicas (también con grandes cantidades de comentario explicativo del funcionamiento) y que tiene que pasar por la clase `MathUtilsTest`. El *bug* introducido en este caso genera errores en una de las 43 pruebas distintas de su batería, cabe destacar; que `MathUtilsTest` hace uso de las funcionalidades de otra clase que debe ser creada localmente `TestUtils` que asiste en la ejecución de las pruebas. Al explorar las diferencias entre versiones de este documento, podemos detectar 10 bloques de diferencias entre ambas, en total, unas 62 líneas diferentes.

4.1.6. Lang 46

La instancia de *Lang 46* refleja sus errores en la clase `StringEscapeUtils` perteneciente a la biblioteca *Apache Commons Math* que añade utilidades de asistencia a la API `java.utils`; cuenta con unas 850 líneas de código físicas y un banco de pruebas en `StringEscapeUtilsTest`. El error reflejado en esta ocasión causa el fallo de uno de sus

casos de prueba, de un total de 18 tests, de forma similar al caso anterior. es necesario la creación de las clases de apoyo `Entities` y `IntHashMap`. Adicionalmente, entre las versiones alternativas de `StringEscapeUtils` existen 8 fragmentos diferenciales, involucrando con ello a 20 líneas en total.

4.1.7. Math 21

El último caso de las instancias de prueba que hemos escogido, *Math 21* reside en el archivo `RectangularCholeskyDecomposition` (también proveniente de *Apache Commons Math*), el cual tiene una métrica LOC de unas 160 líneas, y cuyas pruebas se encuentran en `RectangularCholeskyDecompositionTest`, el cual al ejecutar la versión fallida del documento, obtiene 2 errores de sus 3 pruebas, por lo que es el banco de tests más breve del experimento. En cuanto a las diferencias entre las versiones del archivo, se pueden observar 6 bloques de divergencia, que agrupan 23 líneas en total.

4.1.8. Archivo combinado

La utilización de estos archivos en el proyecto resulta idónea para la prueba de archivos realistas en el contexto del desarrollo software en nuestro sistema de reparación automática. Sin embargo, el espacio de búsqueda que se viaja en cada caso individual resulta aceptable, pero no pone a prueba la capacidad de escala del proceso creado.

Por ello, una adición que realizaremos a la hora de establecer el experimento, es la consideración adicional de todos los bugs como un solo documento. Nuestra intención con este proyecto, es permitir la posibilidad de aplicar sus principios a archivos de todo tipo, por lo que para poder comprobar el potencial en casos más extremos (mayor tamaño de archivo y un gran número de posibles errores), configuraremos el problema y la ejecución del algoritmo para tratar el procesamiento del conjunto de instancias combinado como si fuera una única instancia, tratando la suma de los resultados de los objetivos de cada caso como la solución obtenida por un caso equivalente de mayor tamaño al resto, y considerando un fallo de compilación de uno de los individuos, como un fallo de compilación total de este caso combinado.

Así, en general contamos con una serie *bugs* provenientes de proyectos y archivos variados de distintos tamaños y con características diferentes, así como acciones distintas en las modificaciones realizadas en ellos para arreglarlos. Sin embargo, por motivos que se explicarán más adelante en el documento, tendremos que eliminar a *Lang 46* del caso del archivo combinado, por lo que este estará formado por las otras 6 instancias de prueba. En el Cuadro 1 tenemos un resumen de las propiedades pertinentes de cada uno de las instancias de prueba.

El siguiente paso será elegir los parámetros que se establecerán para la ejecución de los distintos algoritmos en los casos presentados, de vital importancia para la obtención de resultados de calidad.

4.2. Parámetros de los algoritmos

Como hemos establecido, la configuración de los algoritmos utilizados, son de alta importancia: estos parámetros alteran los comportamientos internos de los mismos y pueden afectar tanto de manera positiva como negativa en muchos casos, por lo que es un aspecto al que prestar atención. También como hemos dicho, utilizaremos tres algoritmos para comprobar el sistema construido: **NSGA-II**, **SPEA2** y **Random Search** debido

Cuadro 1: Características de las instancias de prueba utilizadas en el experimento.

Archivo	Origen	Bug ID	LOC	Fragmentos
Calculator	Propio	-	18	9
KeyedObjects2D	Defects4J (Chart)	Chart22	40	5
LevenbergMarquardtOptimizer	Defects4J (Math)	Math64	32	12
MutableDateTime	Defects4J (Time)	Time3	40	10
MathUtils	Defects4J (Math)	Math92	62	10
StringEscapeUtils	Defects4J (Lang)	Lang46	20	8
RectangularCholesky-Decomposition	Defects4J (Math)	Math21	23	6
Caso Combinado	Combinación	-	215	52

a la popularidad y presencia en el campo de los tres. NSGA-II y SPEA2 comparten una gran parte de elementos a configurar, sin embargo, la Búsqueda Aleatoria, debido a su propia naturaleza, no cuenta casi con parámetros: solo necesita el tipo de soluciones a generar y un límite de evaluaciones a hacer, con estos datos podrá generar de forma totalmente aleatoria la cantidad determinada de soluciones para explorar el espacio de las mismas.

Así, haremos uso del framework *jMetal*, que proporciona herramientas para el establecimiento de experimentos con estos tipos de algoritmos y metodologías. Los parámetros se podrían agrupar de la siguiente manera:

4.2.1. Tamaño de la Población, Iteraciones y Evaluaciones

Estos tres elementos, determinan en gran medida el rango de exploración del algoritmo; en otras palabras, establecen en conjunto el número de soluciones que se examinarán y cómo se reproducirán. Los tres en los contextos de los algoritmos que utilizamos, están relacionados mediante la siguiente fórmula:

$$N \cdot I = E_{\max}$$

Donde N es el tamaño de la población, I es el número de iteraciones y $E_{\max}$ el número máximo de evaluaciones. Por tanto, al establecer o modificar uno de ellos, se puede alterar el resto; esto conlleva a que se tenga que elegir una distribución de los valores adecuada para obtener los resultados deseados.

El **tamaño de la población** determina el número de soluciones que se mantienen en cada generación tras el proceso de reproducción, por lo que un número mayor de ellas conlleva una mayor diversidad de soluciones exploradas en cada una de estas iteraciones pero también implica mayor costo computacional. En el proyecto, hemos establecido un tamaño de 20 soluciones en la población.

El **número de iteraciones** representa la cantidad de evoluciones (mediante selección, recombinación y/o mutación) por las que pasará la población. Es decir, los miembros de la población están constantemente en un proceso de evaluación y transformación con el objetivo de producir soluciones de mayor calidad a medida que se avanza de forma simétrica al funcionamiento de la evolución biológica. En nuestro caso, le hemos designado un valor de 50.

Por último, el **número máximo de evoluciones**, como bien indica su nombre, nos señala el límite superior de evaluaciones que se realizarán durante la ejecución general del algoritmo; al realizar ejecuciones de los miembros de la población en cada iteración, podemos ver como este límite viene determinado por los otros dos parámetros. Es probablemente el factor que de forma independiente, indique de forma más directa los recursos y el tiempo que se dedicarán a la realización del experimento; en nuestro caso, cada evaluación implicará construir un archivo, compilarlo y ejecutar sus pruebas. Así, observando los otros valores, podemos ver como este parámetro será de 1000.

4.2.2. Probabilidades de los operadores

En los algoritmos evolutivos, los operadores de recombinación y mutación sirven para evolucionar la población, como se ha establecido. Sin embargo, existe un elemento importante a la hora de involucrarlos en el proceso de optimización: conseguir un balance entre la exploración del espacio de búsqueda y la explotación de las soluciones de buena calidad; para ello, los operadores se ejecutarán con una cierta probabilidad asociada que determinarán el la frecuencia con la que ocurren incrementando además la diversidad y evitando el estancamiento, ya que el procedimiento que se seguirá entre generaciones no será siempre idéntico.

Así, en este proyecto, primeramente hemos designado una probabilidad al operador de cruce de 0.9 (o el 90 %), esto quiere decir que (de forma aproximada) el 90 % de las parejas que sean elegidas para la reproducción generarán sus hijos mediante la recombinación, el resto usarán otros métodos (por ejemplo, simplemente elegir uno de los padres sin modificarlo). Por otro lado, la mutación será un valor relacionado al número de bits con el que cuenten las soluciones del problema, concretamente $1/n$ donde n es este número de bits; de esta forma la mutación se producirá una vez por solución de forma promedia, permitiendo que se ejecute esta mutación en muchos casos, pero no haciendo que sea tan común que introduzca aleatoriedad en las nuevas generaciones.

4.2.3. Ejecuciones independientes

Por último, es necesario establecer un parámetro adicional: en el contexto en el que realizamos estas pruebas, existe un cierto componente de aleatoriedad que va desde la generación de la población inicial (que es aleatoria), hasta las probabilidades de aplicación de los operadores. Por ello, es necesario realizar ejecuciones independientes del mismo algoritmo sobre el mismo problema varias veces, ya que es probable que en ciertas ocasiones los resultados obtenidos puedan ser demasiado negativos o por el contrario, demasiado positivos; al aplicarlos de forma independiente en sobre el mismo caso varias veces podemos tener una visión de los valores promedio que se pueden obtener y por tanto, podemos obtener un grado mayor de fiabilidad de los mismos (además de expandir el número de resultados obtenidos). En este trabajo, se ha establecido un conjunto de 10 ejecuciones de los algoritmos NSGA-II, SPEA2 y Búsqueda Aleatoria sobre los problemas de reparación automática que nos conciernen.

En el Cuadro 2 podemos observar un resumen de los parámetros que se han establecido en las pruebas a realizar. Así, el último paso para poder realizar estos experimentos, será establecer las métricas que nos proporcionarán datos e información que nos interesa sobre los resultados obtenidos.

Cuadro 2: Parámetros generales de los algoritmos utilizados en los experimentos

Parámetro	Valor
Tamaño de la población	20
Número máximo de evaluaciones	1000
Número de generaciones	50
Probabilidad de cruce	0.9
Probabilidad de mutación	$1/n$
Ejecuciones independientes	10

4.3. Métricas utilizadas

A la hora de aplicar estas metodologías, es posible obtener ciertas medidas o métricas de los resultados obtenidos tras la ejecución de los algoritmos o incluso datos recabados durante el proceso de ejecución. Principalmente, nos servirán para evaluar el desempeño de estas metodologías aplicadas a nuestros casos, y para ello utilizaremos tres métricas principales: **Hipervolumen (HV)**, **Spread** y **Tiempo de ejecución**. Con ellas, esperamos poder analizar la calidad de las soluciones generadas y la eficiencia computacional del sistema creado y la aplicación de los algoritmos a este.

4.3.1. Hipervolumen (Hypervolume, HV)

El hipervolumen mide el tamaño del espacio objetivo que está dominado por un conjunto de soluciones respecto a un punto de referencia cuya elección es de gran importancia; en otras palabras, es el volumen de un espacio donde cualquier punto está dominado por al menos una de las soluciones. En problemas de 2 objetivos como el nuestro, se podría también decir que es el área bajo la curva formada por el frente de Pareto hasta llegar al punto de referencia escogido. Cuanto mayor es el valor del hipervolumen, más cerca está el frente aproximado del frente de Pareto óptimo y más diversidad de soluciones contiene este frente aproximado, ya que cubre una zona mayor. Por lo tanto, este valor nos indicará la efectividad a la hora de equilibrar los objetivos perseguidos. En la Figura 4 podemos ver una ilustración que ejemplifica la métrica.

Cabe señalar que para reflejar el desempeño de los algoritmos, utilizaremos un hipervolumen normalizado, de manera que sea más sencillo comprobar la eficiencia a partir de esta métrica. En el contexto de *jMetal* el hipervolumen normalizado se calcula restandole a un valor 1 la división entre el hipervolumen frente de soluciones obtenido, y el de referencia (que sería el mejor conjunto de soluciones que se conoce que se puede obtener). Por tanto, cuanto más se acerque a 0 este valor, más cercano a ser óptimo estará el frente obtenido con respecto a los resultados totales.

4.3.2. Spread

Otra métrica clásica y altamente utilizada en el análisis de la optimización multiobjetivo es el spread (también llamado Dispersión) y muchas veces usada como complemento del hipervolumen. Mientras que el hipervolumen nos da una medida de la relativa calidad del conjunto de soluciones, el spread nos ayuda a medir la optimización de la distribución de las soluciones a lo largo del conjunto de Pareto. A la hora de elegir soluciones para poblar el frente de Pareto, es posible que el conjunto acabe siendo individuos que, aunque no dominados, representan cambios muy pequeños, por lo tanto, cuanto más distintas

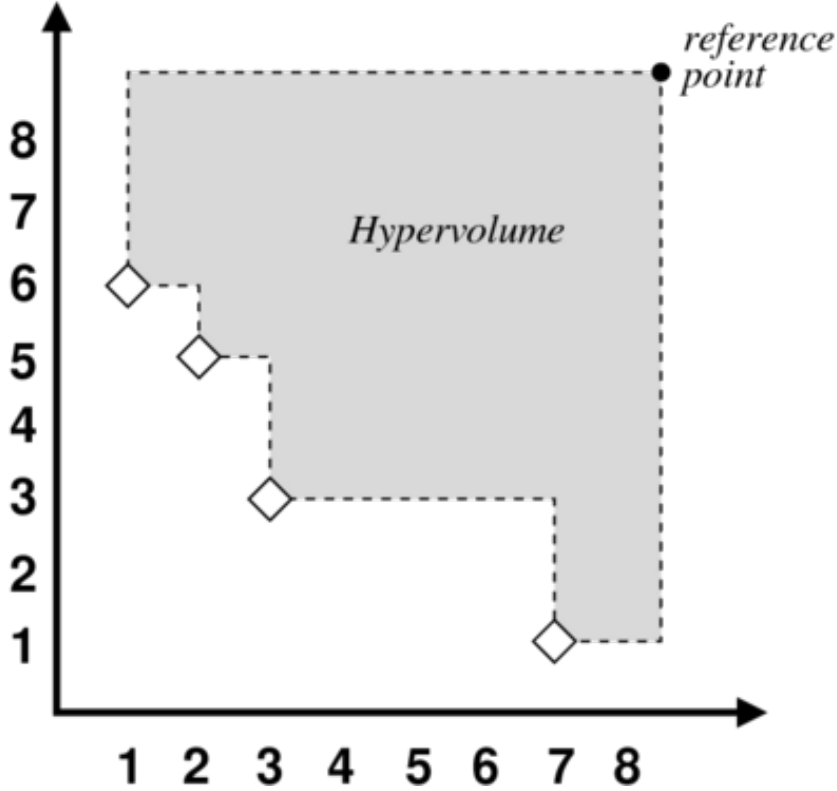


Figura 4: Ejemplo ilustrativo del cálculo del hipervolumen en optimización multiobjetivo [9].

sean las soluciones que no estén dominadas entre sí, más diverso es el conjunto de soluciones y por tanto, más interesantes y significativas serán las elecciones entre los miembros del frente.

La idea que empuja a la teoría detrás de esta métrica, en general, sería que las distancias mínimas y máximas de cada par de soluciones consecutivas (según un objetivo u orden preestablecido) fueran similares a las distancia media entre todos los individuos, lo cual implica una buena distribución. En nuestro caso, implicaría documentos con distintos balances entre los números de errores y los fragmentos de código modificados. Específicamente, en nuestro caso, utilizamos la misma definición de la métrica establecida en [4] donde, resumidamente:

- Se calculan las distancias entre los extremos de las listas ordenadas (por cualquier medio escogido) del frente de Pareto obtenido y el frente de referencia.
- Se calculan las distancias entre cada par de soluciones consecutivas del frente obtenido y se genera media de dichos valores.
- Se suman las distancias entre los extremos y las desviaciones de las distancias entre soluciones consecutivas de la media.
- Se normaliza el resultado

Este proceso viene definido por la fórmula siguiente también incluida en [4]:

$$\Delta = \frac{d_f + d_l + \sum_{i=1}^{N-1} |d_i - \bar{d}|}{d_f + d_l + (N - 1)\bar{d}} \quad (1)$$

4.3.3. Tiempo de ejecución

Una medida directa y sencilla, el tiempo de ejecución nos puede dar una cierta medición sobre la efectividad o complejidad del problema, sin embargo, el hecho de cambiar máquinas entre ejecuciones, el uso de diferentes núcleos para las ejecuciones y el factor de la aleatoriedad pueden alterar considerablemente el tiempo empleado en el proceso, por lo que puede ser de utilidad, pero teniendo en cuenta estas variables; por ello incluiremos unos valores medios aproximados de los tiempos de ejecución como referencia.

5. Análisis de los Resultados

Así, una vez hemos establecido la teoría detrás de nuestro sistema de optimización multiobjetivo de las posibles combinaciones de distintas versiones de un archivo, hemos escogido los algoritmos para la asistencia en su creación y hemos determinado los parámetros bajo los que las comprobaciones serán ejecutadas así como los resultados que queremos obtener de las mismas, debemos proceder a realizar los experimentos. Para ello, utilizaremos *jMetal* y las capacidades de configuración de experimentos que nos otorga, entre otras cosas, la posibilidad de crear un experimento general con todos los problemas e instancias que se desea. En nuestro caso, el primer experimento que crearemos será el de los ficheros individuales.

5.1. Experimento 1: Ficheros individuales

El primer paso será construir los casos explorados de forma individual y comprobar los resultados de la ejecución del sistema en cada uno de estos observando las métricas producidas. Este experimento nos servirá para validar el correcto funcionamiento del acercamiento en primera instancia sobre casos más sencillos. Primeramente, utilizaremos los fragmentos de código diferencial como unidad de división y de creación del documento resultado.

5.1.1. Análisis por bloques

El análisis por bloques de los archivos individuales es probablemente, el experimento en el que mejores resultados ha tenido el sistema en cuestión de encontrar los archivos que mejor combinen los resultados de las pruebas y la minimización del uso de fragmentos del archivo original; lo cual resulta lógico gracias a las dimensiones menores de su espacio de búsqueda, en el mayor de los casos, el vector binario tendrá unos 12 bits de tamaño lo cual aunque relativamente extenso, no resulta excesivo para el experimento configurado.

Si observamos primero el Cuadro 3 se pueden observar los valores medios para el hipervolumen normalizado de cada una de las instancias en los algoritmos. Como hemos establecido, es un hipervolumen en el que cuanto mayor se acerque a cero, más cercanos han sido los resultados de los objetivos al frente de Pareto generado durante la ejecución; este, en el caso de *Defects4J*, será el valor mínimo total (0,0); mientras que en el caso *Calculator* el frente es más variado por las características del mismo.

Así, como es evidente en la tabla, la gran mayoría de las instancias han conseguido encontrar el valor óptimo general que se ha asignado tras la exploración; como se ha dicho anteriormente, tiene sentido pues los parámetros del experimento permiten la exploración exhausta del espacio de soluciones. El único algo divergente es en *Lang 46* en los que

Instancia	NSGA-II	SPEA2	RS
Calculator	$0.00e + 00_{0.0e+00}$	$0.00e + 00_{0.0e+00}$	$0.00e + 00_{0.0e+00}$
Chart 22	$0.00e + 00_{0.0e+00}$	$0.00e + 00_{0.0e+00}$	$0.00e + 00_{0.0e+00}$
Math 64	$0.00e + 00_{0.0e+00}$	$0.00e + 00_{0.0e+00}$	$0.00e + 00_{0.0e+00}$
Time 3	$0.00e + 00_{0.0e+00}$	$0.00e + 00_{0.0e+00}$	$0.00e + 00_{0.0e+00}$
Math 92	$0.00e + 00_{0.0e+00}$	$0.00e + 00_{0.0e+00}$	$0.00e + 00_{0.0e+00}$
Lang 46	$4.00e - 01_{5.2e-01}$	$0.00e + 00_{0.0e+00}$	$0.00e + 00_{0.0e+00}$
Math 21	$0.00e + 00_{0.0e+00}$	$0.00e + 00_{0.0e+00}$	$0.00e + 00_{0.0e+00}$

Cuadro 3: Valores medio con desviación estándar del hipervolumen normalizado para cada instancia en cada algoritmo en el análisis por fragmentos

algunas de las ejecuciones no encontraron su valor óptimo, concretamente en NSGA-II; en este caso encuentra soluciones cercanas al óptimo, pero debido a su comparación con el valor mínimo obtenido en otras iteraciones, su hipervolumen normalizado es 1 y la media de su algoritmo refleja este hecho.

En general, el hipervolumen normalizado muestra como la gran mayoría de las ejecuciones han obtenido los valores óptimos para sus objetivos, y, exceptuando cierta anomalía en un caso concreto, demuestra la viabilidad de la optimización en estos casos.

Instancia	NSGA-II	SPEA2	RS
Calculator	$1.57e - 16_{0.0e+00}$	$1.26e + 00_{1.1e-16}$	$4.29e - 02_{6.9e-02}$
Chart 22	$1.00e + 00_{0.0e+00}$	$1.00e + 00_{0.0e+00}$	$1.00e + 00_{0.0e+00}$
Math 64	$1.00e + 00_{0.0e+00}$	$1.00e + 00_{0.0e+00}$	$1.00e + 00_{0.0e+00}$
Time 3	$1.00e + 00_{0.0e+00}$	$1.00e + 00_{0.0e+00}$	$1.00e + 00_{0.0e+00}$
Math 92	$1.00e + 00_{0.0e+00}$	$1.00e + 00_{0.0e+00}$	$1.00e + 00_{0.0e+00}$
Lang 46	$1.00e + 00_{0.0e+00}$	$1.00e + 00_{0.0e+00}$	$1.00e + 00_{0.0e+00}$
Math 21	$1.00e + 00_{0.0e+00}$	$1.00e + 00_{0.0e+00}$	$1.00e + 00_{0.0e+00}$

Cuadro 4: Valores medio con desviación estándar del Spread para cada instancia en cada algoritmo en el análisis por fragmentos

Por otra parte, si observamos los valores obtenidos por la dispersión de los resultados, vemos como en la gran mayoría de los resultados, se obtiene el peor valor de la métrica: 1. Esto es debido también a la naturaleza de los documentos de *Defects4J*, los resultados que hemos visto en el hipervolumen y la búsqueda en general planteada: al ser estos casos problemas en los que la solución de minimización de todos los objetivos es posible, y al ser esta encontrada por la gran mayoría de las ejecuciones (o encontrar una única solución muy cercana a la óptima), en todos estos casos, la dispersión de un único punto en el espacio de búsqueda es 1, su peor valor y el que indica que los puntos estarían concentrados.

Sin embargo, en este caso, el caso divergente es la calculadora; en la que caben destacar dos cosas: el primero es el valor en SPEA2, esto es debido a la integración del algoritmo en *jMetal* el cual en su archivo de soluciones final, puede repetir soluciones, lo cual se da en casos como el de la calculadora haciendo que este valor sea incluso mayor que 1, lo cual no debería ser posible. En realidad, eliminando casos repetidos, la dispersión de la calculadora en SPEA2 es más parecida a la de NSGA-II. Así, sus valores, se acercan bastante a este valor idóneo del Spread, lo que indica una buena dispersión de los valores obtenidos en el espacio de búsqueda.

Adicionalmente, en el Cuadro 5 podemos observar los mejores valores encontrados por cada algoritmo en las respectivas instancias; en la mayoría de ellos se ha obtenido el mejor punto $(0, 0)$ exceptuando *Calculator*, el cual ha obtenido una distribución de puntos óptimos para su instancia, por lo que podemos decir que $(0, 0)$ no es posible en su caso.

Instancia	NSGA-II	SPEA2	RS
Calculator	(7, 0) (0, 7) (1, 6) (6, 1) (2, 5) (4, 3) (3, 4) (5, 2)	(7, 0) (0, 7) (1, 6) (6, 1) (2, 5) (4, 3) (3, 4) (5, 2)	(7, 0) (0, 7) (1, 6) (6, 1) (2, 5) (4, 3) (3, 4) (5, 2)
Chart 22	(0, 0)	(0, 0)	(0, 0)
Math 64	(0, 0)	(0, 0)	(0, 0)
Time 3	(0, 0)	(0, 0)	(0, 0)
Math 92	(0, 0)	(0, 0)	(0, 0)
Lang 46	(0, 0)	(0, 0)	(0, 0)
Math 21	(0, 0)	(0, 0)	(0, 0)

Cuadro 5: Mejores valores obtenidos en cada instancia y algoritmo en el análisis por fragmentos. Se resaltan las soluciones no dominadas en su instancia

Tomando todos los datos en conjunto, podemos ver como, en general, los algoritmos planteados encuentran la solución única en los casos de *Defects4J*, obteniendo por ello el mayor valor del hipervolumen normalizado y al mismo tiempo, causando que la dispersión sea el peor valor posible de 1. En el caso de la calculadora, esta presenta datos buenos y más dispersos que los demás, al contar en sus soluciones con un grupo de individuos.

5.1.2. Análisis por líneas

Por otra parte, al realizar el análisis de las instancias por líneas, podemos ver las limitaciones del acercamiento planteado problemas presentes a la hora de capturar las métricas con los objetivos planteados. Al tratar con casos de mayor extensión que los problemas por fragmentos, el primero de los problemas surge: es más difícil conseguir soluciones, y más incluso soluciones de alta calidad. Este es primeramente un gran obstáculo en nuestra experimentación: los costes computacionales de la constante generación, compilación y comprobación de los documentos tienen exigen unos requisitos incompatibles con las necesidades temporales del proyecto en cierta medida.

Esta limitación de soluciones encontradas, se extiende también a la generación de las métricas vitales para la comparación. En el uso de estas métricas en problemas donde los frentes de referencia son desconocidos a priori, se requiere de la creación de uno a partir de los resultados obtenidos. Sin embargo, en el caso del sistema por líneas la mayor capacidad de especificación en el documento amplía en gran medida el número de versiones no compilables; esto junto con un aumento del espacio de búsqueda complica el proceso de obtención de soluciones al menos viables. Esto, junto con la naturaleza de la mayoría de nuestros problemas, en los que muchas veces los objetivos no se contradicen en muchos casos (sobre todo en *Defects4J*) lleva a dos posibles sucesos:

- No se encuentra ninguna solución válida
- El frente de referencia sea una única solución distinta del mínimo total $(0,0)$

Estos factores, llevan a un cálculo imposible de las métricas o a medidas erróneas o fallidas en la gran mayoría de los casos, por lo que no es útil la utilización de las mismas para comparar su rendimiento. Para ello, recogemos en su lugar el desempeño final de cada algoritmo en el Cuadro 6, reflejando la mejor solución obtenida por cada algoritmo en cada instancia, siendo el primer número la cantidad de pruebas fallidas y el segundo, el número de líneas del archivo original utilizadas, marcando la casilla con el color rojo si la solución obtenida es no compilable:

Instancia	NSGA-II	SPEA2	RS
Calculator	(1, 6) (3, 4) (2, 5)	(4, 3)	(5, 2)
Chart 22	(11, 5)	(11, 5)	(11, 5)
Math 64	(0, 0)	(0, 0)	(18, 0) (0, 4) (15, 2) (2, 3)
Time 3	(12, 0) (3, 1)	(2, 0)	(5, 4)
Math 92	(43, 11)	(43, 11)	(43, 11)
Lang 46	(1, 0)	(2, 5) (1, 6)	(18, 9)
Math 21	(0, 0)	(2, 0)	(3, 3) (2, 9)

Cuadro 6: Mejores valores obtenidos en cada instancia y algoritmo en el análisis por líneas. Se resaltan las soluciones no dominados en su instancia, peores resultados posibles en rojo

Así, como podemos observar, solo en 3 pares de algoritmo-instancia de prueba se han encontrado el óptimo total, mientras que existen una presencia importante de ejecuciones de algoritmo sobre instancia que no llegan a la solución, adicionalmente, dos de las instancias de prueba no consiguen ninguna solución válida y una falla en las ejecuciones de la búsqueda aleatoria. Adicionalmente, por lo general NSGA-II y SPEA2 han podido encontrar soluciones de forma más óptima que la búsqueda aleatoria, ya que podemos observar que las soluciones de ambos (exceptuando la calculadora) siempre dominan a este último. Al mismo tiempo, el desempeño de NSGA-II parece ligeramente mejor al de SPEA2, con una mayor variedad en la calculadora y mejores resultados en *Lang 46* y *Math 21*, aunque siendo peor en *Time 3*.

Como hemos establecido, la aparición de un mayor número de posibles soluciones, así como el nivel de detalle del sistema por líneas, hace que resulte más complicado encontrar los valores óptimos de las líneas de diferencia, pero incluso en situaciones limitadas, podemos observar como este sistema sigue encontrando individuos viables y en algunos incluso, la mejor opción posible. Esto, muestra que la aplicación del sistema a la división por líneas podría beneficiarse de un mayor recurso temporal en su ejecución y con ello, podría encontrarse un documento final que pasara correctamente las pruebas. Esto es además aplicable a los ejemplos de *Defects4J* y *Calculator* a pesar de sus perfiles ligeramente distintos; lo cual demuestra también una adaptación a las condiciones del archivo.

5.1.3. Comparación

De esta forma, observando los resultados de los valores objetivo en las soluciones obtenidas en cada uno de los métodos de división, podemos ver como los resultados

finales bajo las condiciones establecidas son mucho mejores en el caso de la división por fragmentos que al dividir por líneas. Esto resulta lógico por dos motivos:

- La división por líneas aumenta el espacio de búsqueda, por lo tanto requiere de mayor esfuerzo computacional y requisitos temporales
- La división por líneas trata la generación del documento resultado con mayor especificidad, facilitando la creación de individuos que fallen la compilación

Estos dos factores llevan a una complicación mayor a la hora de encontrar soluciones válidas: es mucho más fácil generar código erróneo mediante la división por líneas que por fragmentos, en los cuales factores como la finalización errónea de estructuras no es tan común. Sin embargo, podemos ver al mismo tiempo, como en muchas instancias, la división por líneas ha podido minimizar por completo o parcialmente los objetivos, lo cual demuestra que en condiciones apropiadas, puede realizar las mismas tareas que la diferenciación por fragmentos.

Por lo tanto, podemos ver como en los casos individuales, la aplicación de ambas formas de estructurar las diferencias entre archivos, pueden aportar resultados óptimos, pero la división de cambios por líneas, necesita de un mayor esfuerzo computacional que la metodología por fragmentos, y en caso de contar con ella, se pueden generar archivos de forma más específica; esto puede ser beneficioso en casos en los que la construcción del software necesita ser más detallada.

5.2. Experimento 2: Fichero combinado

Así, una vez hemos realizado las comprobaciones sobre archivos individuales, pasamos a comprobar la experimentación sobre la versión del problema en la que se combinan todos los archivos estudiados como si fuera una única instancia de prueba. Sin embargo, al realizar las comprobaciones de sus resultados, encontramos un contratiempo: ninguna de las versiones era capaz de generar una solución válida tras la ejecución de todos los algoritmos; lo cual es un problema, pues no podemos comprobar su rendimiento de ninguna forma (métricas o resultados).

Por ello, realizamos una examinación de los archivos individuales y sus combinaciones en una metodología de división por fragmentos. Esto, nos llevó a descubrir que el descarte del caso *Lang 46* permite que se obtengan resultados válidos en ejecuciones del experimento combinado por fragmentos, así, para poder realizar un mínimo análisis de la ejecución del sistema sobre un caso más grande, debemos eliminar esta instancia de prueba de la versión del experimento en un fichero combinado; esto por supuesto no elimina la validez del caso de forma individual, pero las necesidades temporales del proyecto llevan a esta decisión para poder obtener resultados significativos

5.2.1. Análisis por bloques

El análisis por bloques del fichero combinado, resulta consistente en sus resultados con las ejecuciones individuales: el aumentar el espacio de búsqueda hace que el número de soluciones compilables y la calidad de las mismas se reduzcan con respecto a los casos más pequeños; sin embargo, al utilizar la metodología por fragmentos, las posibilidades de archivos no compilables, no son tan altas; por lo que existe un equilibrio entre ambos factores.

Métrica	NSGA-II	SPEA2	RS
Hipervolumen Normalizado	$9.06e - 01_{2.0e-01}$	$8.09e - 02_{8.1e-02}$	$1.00e + 00_{0.0e+00}$
Spread	$9.67e - 01_{4.2e-02}$	$1.19e + 00_{1.1e-01}$	$7.21e - 01_{6.4e-02}$

Cuadro 7: Valores medio con desviación estándar del hipervolumen normalizado y Spread en el archivo combinado por fragmentos

Como podemos observar en el Cuadro 7, el algoritmo con un mejor rendimiento en este caso ha sido SPEA2, con respecto a las soluciones obtenidas a lo largo del experimento; su hipervolumen normalizado en general está cerca del valor 0, mientras que su dispersión presenta el mismo error que en el caso individual de la calculadora, debido a la repetición de soluciones en su archivo. Por otra parte NSGA-II ha conseguido soluciones aceptables en ciertas ejecuciones, pero ha sido mucho menos consistente que SPEA2, mientras que su dispersión tampoco ha sido la mejor, siendo cercana al valor 1. La búsqueda aleatoria, ha obtenido resultados consistentemente, pero sus soluciones encontradas por lo general, no son igual de óptimas que las de los otros algoritmos; a pesar de ello, su dispersión es mejor que la de NSGA-II denotando una mayor variedad en su frente de soluciones.

Instancia	NSGA-II	SPEA2	RS
Caso Combinado	(2, 20) (3, 18)	(10, 9) (6, 13) (9, 10) (7, 12) (8, 11) (4, 15) (5, 14)	(14, 17) (9, 24) (26, 16)

Cuadro 8: Mejores valores obtenidos en cada algoritmo en el análisis por fragmentos del archivo combinado. Se resaltan las soluciones no dominadas

Estos factores pueden comprobarse también en el Cuadro 8, donde podemos encontrar las mejores soluciones obtenidas por cada algoritmo en este caso, como podemos ver, tanto NSGA-II como SPEA2 encuentran soluciones no dominadas entre sí, por lo que ambos han logrado generar documentos óptimos en sus ejecuciones que el otro no ha conseguido. Por otra parte, la búsqueda aleatoria ha encontrado soluciones pero ninguna de ellas mejor que las de los otros dos algoritmos que se benefician de una metodología establecida por encima de la aleatoriedad.

Así, observando el documento combinado, podemos establecer que los tres algoritmos han conseguido la obtención de resultados, cada uno mediante sus métodos establecidos y con dispersión e hipervolumen distintos; esto demuestra que en las condiciones ligeramente limitadas del experimento, el sistema de reparación automática ha podido realizar su cometido hasta cierto punto, probando que también funciona en contextos de espacios de búsqueda mayores.

5.2.2. Análisis por líneas

Por último, en el caso del archivo combinado por líneas, sucede una situación similar a la de los casos individuales, pero más concreta: este problema es el más extenso de todos y además el más propenso a generar soluciones no compilables; la combinación de ambos factores lleva a que en el contexto de nuestra experimentación, sea imposible conseguir una solución que no sea la peor posible en el problema y por lo tanto sea no se pueden recabar las métricas de esta ejecución y en todo caso no serían de utilidad. Concretamente, los resultados obtenidos en todas las ejecuciones son el valor (125, 54).

Así, este es el experimento en el que más sería necesario ampliar los límites de la experimentación con el objetivo de poder asegurar el funcionamiento correcto del sistema en un caso extremo como este.

5.2.3. Comparación

En consecuencia, debido en parte a las limitaciones del experimento, la comparación de ambos casos beneficia claramente al análisis por fragmentos, al haber sido capaz de generar resultados al menos admisibles (y cerca del óptimo en muchos casos); sin embargo, esto no significa que el sistema no pueda aplicarse al mismo, las aplicaciones y resultados positivos tanto en el caso combinado por fragmentos y los casos individuales por líneas, sugieren que la reparación automática como ha sido planteada, podría aplicarse a este caso de mayores dimensiones por líneas, lo cual abre la puerta a futuros desarrollos del mismo.

Como último apunte, cabe destacar que un elemento importante que se pretendía reportar es el tiempo de ejecución de los algoritmos; sin embargo, las ejecuciones de los mismos requirieron un tiempo considerable y no se disponía de un equipo exclusivo para dedicado a realizarlas, por lo que estos tiempos se vieron afectados por otros procesos. Aun así, se pueden obtener una serie de conclusiones interesantes sobre este aspecto:

- La evaluación de las soluciones es el proceso más costoso del algoritmo con diferencia lo cual causaba que el tiempo final dependiera en gran medida al número de evaluaciones. Por ello, el tiempo de ejecución era similar en todos los algoritmos, ya que los costes temporales de los funcionamientos internos de estos se veían ocultos por la evaluación.
- De forma contradictoria a la teoría, el aumento de la complejidad a la hora de resolver el problema, implicaba un menor tiempo de cómputo. Esto es debido a que en estos tipos de casos, un número considerable de las soluciones no eran compilables y por tanto, no se ejecutaban las pruebas, reduciendo el tiempo de evaluación.

6. Conclusiones y trabajos futuros

En este trabajo se ha desarrollado un proyecto de investigación sobre la **reparación automática de errores en programas Java**, en el contexto de integración continua y automatización del ciclo de desarrollo. El objetivo fue evaluar hasta qué punto técnicas de optimización multiobjetivo permiten generar y validar parches que reduzcan el esfuerzo de los desarrolladores y mejoren la calidad del software.

6.1. Resumen del trabajo realizado

Se revisó el estado del arte en CI/CD y reparación automática de programas, identificando enfoques como GenProg [16] y ARJA [27], que utilizan métodos de búsqueda y optimización guiados por pruebas unitarias estudiando además, herramientas como *jMetal*, las cuales resultan útiles para implementar metaheurísticas de optimización multiobjetivo.

Una vez revisado el trabajo en el campo, se procedió a plantear el problema. De esta forma, el proyecto se planteó como la combinación de dos versiones de un archivo, generando candidatos que representan posibles parches. Se propusieron dos modelos de

representación mediante vectores binarios: por segmentos y por líneas. Los objetivos fueron maximizar la corrección (minimizar las pruebas fallidas) y minimizar la cantidad de fragmentos tomados del archivo original, con la meta de realizar una comprobación de del grado de corrección de un archivo nuevo generado a partir de otro, a modo de parche.

Para resolver el problema se usaron los algoritmos NSGA-II, SPEA2 y Random Search, junto con operadores clásicos de selección, recombinación y mutación. La implementación se realizó en Java, apoyada en *jMetal*, *Java Diff Utils* y *JUnit*.

El diseño experimental incluyó un caso de prueba creado de forma personalizada y ejemplos del dataset *Defects4J*, en total 7 instancias de prueba para el experimento pertinente. Se intentó medir el rendimiento mediante métricas como hipervolumen y dispersión, y se establecieron un número total de evaluaciones de acuerdo a las complejidades computacionales del proyecto. Uno de los casos (*Lang 46*) debió ser descartado de la versión combinatoria posteriormente por su complejidad a la hora de mostrar resultados relevantes.

La ejecución del experimento muestra una serie de factores:

- La reparación por segmentos funciona de forma efectiva en casos individuales, aunque se ve limitada en instancias combinadas.
- La reparación por líneas encuentra algunas soluciones válidas, pero su efectividad es baja en espacios de búsqueda amplios.
- La recolección de métricas choca con los casos establecidos y los objetivos seleccionados

Así, los experimentos a baja escala muestran un funcionamiento correcto y eficiente con respecto a las soluciones encontradas, e incluso al tratar con una búsqueda más específica y sensible como es la división por líneas, es posible encontrar soluciones que minimicen hasta cierto punto los objetivos buscados. Sin embargo, como es lógico, en casos más extensos y complejos las limitaciones del experimento resultan más aparentes; pero los casos positivos de su aplicación demuestran potencial para expandir su funcionamiento y adaptarlo a casos más grandes. Adicionalmente, los objetivos impuestos y sus resultados pueden chocar con la obtención de métricas lo cual podría ser un problema en futuros desarrollos.

Observando el trabajo en general, podemos decir que se ha podido realizar la implementación de un prototipo funcional de reparación automática de errores en Java, mediante la evaluación de distintas versiones de código por medio de pruebas sobre los mismos, y una comprobación de su aplicación en casos realistas. Sin embargo, pese a los logros alcanzados, se produce una dependencia única en estas pruebas en gran medida, lo cual puede ser defectivo y el coste computacional y requisitos temporales del proyecto han limitado la extensión y por tanto la aplicación de la experimentación sobre el sistema, por lo que este aspecto podría ser expandido.

6.2. Líneas futuras

A partir del proyecto realizado y la experiencia obtenida con ello, además de las limitaciones que han ocurrido, se plantean distintas líneas de trabajo futuro:

1. **Integración de técnicas de aprendizaje automático:** explorar el uso de modelos de lenguaje y algoritmos de aprendizaje supervisado o reforzado que puedan guiar la generación de parches hacia soluciones más prometedoras.

2. **Mejora de la validación:** complementar las pruebas unitarias con otros objetivos cuantificables sobre los documentos tratados o de otro ámbito
3. **Optimización del rendimiento:** investigar mecanismos para la agilización de los procesos de forma que permitan aplicar las técnicas de reparación en proyectos de gran escala sin penalizar la velocidad del ciclo CI/CD.
4. **Extensión a otros lenguajes de programación:** adaptar el prototipo a entornos distintos de Java (por ejemplo, Python, JavaScript o C++), lo que ampliaría su aplicabilidad en contextos industriales.

En conclusión, los resultados alcanzados muestran que, como ampliamente se conoce en los contextos software, la reparación automática de programas es una línea de investigación con gran potencial de impacto en la práctica de la ingeniería del software. El trabajo desarrollado sienta una base sólida con limitaciones y aspectos prometedores, sobre la que profundizar con nuevas técnicas, mayor validación empírica y una integración más estrecha con las herramientas de desarrollo contemporáneas.

Referencias

- [1] Alberto Bacchelli and Christian Bird. Expectations, outcomes, and challenges of modern code review. In *2013 35th International Conference on Software Engineering (ICSE)*, pages 712–721. IEEE, 2013.
- [2] Julian Blank and Kalyanmoy Deb. Pymoo: Multi-objective optimization in python. *Ieee access*, 8:89497–89509, 2020.
- [3] Sébastien Cahon, Nordine Melab, and E-G Talbi. Paradiseo: A framework for the reusable design of parallel and distributed metaheuristics. *Journal of heuristics*, 10(3):357–380, 2004.
- [4] Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and TAMT Meyarivan. A fast and elitist multiobjective genetic algorithm: Nsga-ii. *IEEE transactions on evolutionary computation*, 6(2):182–197, 2002.
- [5] Thomas Durieux, Fernanda Madeiral, Matias Martinez, and Rui Abreu. Empirical review of java program repair tools: A large-scale experiment on 2,141 bugs and 23,551 repair attempts. In *ESEC/FSE 2019 - Proceedings of the 2019 27th ACM Joint Meeting European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, page 302 – 313, 2019.
- [6] Juan J. Durillo and Antonio J. Nebro. Jmetal: A java framework for multi-objective optimization. *Advances in Engineering Software*, 42(10):760 – 771, 2011.
- [7] Paul M Duvall, Steve Matyas, and Andrew Glover. *Continuous integration: improving software quality and reducing risk*. Pearson Education, 2007.
- [8] Omar Elazhary, Colin Werner, Ze Shi Li, Derek Lowlind, Neil A Ernst, and Margaret-Anne Storey. Uncovering the benefits and challenges of continuous integration practices. *IEEE Transactions on Software Engineering*, 48(7):2570–2583, 2021.
- [9] Carlos M. Fonseca, Manuel López-Ibáñez, Luís Paquete, and Andreia P. Guerreiro. Computation of the hypervolume indicator. <https://lopez-ibanez.eu/hypervolume>. Accedido por última vez: 2025-09-01.
- [10] Félix-Antoine Fortin, François-Michel De Rainville, Marc-André Gardner Gardner, Marc Parizeau, and Christian Gagné. Deap: Evolutionary algorithms made easy. *The Journal of Machine Learning Research*, 13(1):2171–2175, 2012.
- [11] Free Software Foundation. diff: File comparison utility, 2023. Accedido por última vez: 2025-09-02.
- [12] Martin Fowler and Matthew Foemmel. Continuous integration, 2006.
- [13] Jez Humble and David Farley. *Continuous delivery: reliable software releases through build, test, and deployment automation*. Pearson Education, 2010.
- [14] René Just, Darioush Jalali, and Michael D Ernst. Defects4j: A database of existing faults to enable controlled testing studies for java programs. In *Proceedings of the 2014 international symposium on software testing and analysis*, pages 437–440, 2014.

- [15] D. Kerr et al. Javadiffutils: Java diff utility, 2023. Accedido por última vez: 2025-09-02.
- [16] Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. Genprog: A generic method for automatic software repair. *Ieee transactions on software engineering*, 38(1):54–72, 2011.
- [17] Arnaud Liefooghe, Laetitia Jourdan, Thomas Legrand, Jérémie Humeau, and El-Ghazali Talbi. Paradiseo-moeo: A software framework for evolutionary multi-objective optimization. In *Advances in Multi-Objective Nature Inspired Computing*, pages 87–117. Springer, 2010.
- [18] Kui Liu, Shangwen Wang, Anil Koyuncu, Kisub Kim, Tegawende F. Bissyande, Dongsun Kim, Peng Wu, Jacques Klein, Xiaoguang Mao, and Yves Le Traon. On the efficiency of test suite based program repair a systematic assessment of 16 automated repair systems for java programs. In *Proceedings - International Conference on Software Engineering*, page 615 – 627, 2020.
- [19] Yuzhen Liu, Long Zhang, and Zhenyu Zhang. A survey of test based automatic program repair. *J. Softw.*, 13(8):437–452, 2018.
- [20] Martin Monperrus. Automatic software repair: A bibliography. *ACM Computing Surveys*, 51(1), 2019.
- [21] Antonio J. Nebro, Cristóbal Barba-González, Manuel López-Ibáñez, and José García-Nieto. Automatic configuration of nsga-ii with jmetal and irace. In *GECCO 2019 Companion - Proceedings of the 2019 Genetic and Evolutionary Computation Conference Companion*, page 1374 – 1381, 2019.
- [22] Yuhua Qi, Xiaoguang Mao, Yan Lei, Ziyang Dai, and Chengsong Wang. The strength of random search on automated program repair. In *Proceedings - International Conference on Software Engineering*, number 1, page 254 – 265, 2014.
- [23] Mojtaba Shahin, Muhammad Ali Babar, and Liming Zhu. Continuous integration, delivery and deployment: a systematic review on approaches, tools, challenges and practices. *IEEE access*, 5:3909–3943, 2017.
- [24] Westley Weimer, ThanhVu Nguyen, Claire Le Goues, and Stephanie Forrest. Automatically finding patches using genetic programming. In *2009 IEEE 31st International Conference on Software Engineering*, page 364 – 374, 2009.
- [25] Jifeng Xuan, Matias Martinez, Favio DeMarco, Maxime Clement, Sebastian Lamelas Marcote, Thomas Durieux, Daniel Le Berre, and Martin Monperrus. Nopol: Automatic repair of conditional statement bugs in java programs. *IEEE Transactions on Software Engineering*, 43(1):34 – 55, 2017.
- [26] Jinqiu Yang, Alexey Zhikhartsev, Yuefei Liu, and Lin Tan. Better test cases for better automated program repair. In *Proceedings of the ACM SIGSOFT Symposium on the Foundations of Software Engineering*, volume Part F130154, page 831 – 841, 2017.

- [27] Yuan Yuan and Wolfgang Banzhaf. Arja: Automated repair of java programs via multi-objective genetic programming. *IEEE Transactions on Software Engineering*, 46(10):1040–1067, 2018.
- [28] Eckart Zitzler, Marco Laumanns, and Lothar Thiele. Spea2: Improving the strength pareto evolutionary algorithm. *TIK report*, 103, 2001.



UNIVERSIDAD
DE MÁLAGA

| **uma.es**

E.T.S. DE INGENIERÍA INFORMÁTICA

E.T.S de Ingeniería Informática
Bulevar Louis Pasteur, 35
Campus de Teatinos
29071 Málaga